

01

Advanced DevOps Engineering

Production-Grade DevOps Systems

IN THIS WEEK · 4 TOPICS · 72 SECTIONS

01	Advanced Git & CI/CD Engineering ADVANCED GIT ENGINEERING Git Internals & Object Model Branching Strategies
02	Docker & Container Security Engineering Containerization Fundamentals Dockerfile Best Practices Multi-Stage Builds
03	Kubernetes Production Concepts Kubernetes Architecture Overview Control Plane Components Worker Node Components
04	Observability & Monitoring Observability Principles Metrics vs Logs vs Traces Golden Signals

WEEK 01 OF 17

Advanced Git & CI/CD Engineering

ADVANCED GIT ENGINEERING

01 Git Internals & Object Model

Most engineers use Git every day without understanding how it actually works internally. This is fine for basic usage, but when things go wrong — a rebase goes sideways, a reset loses commits, a merge produces unexpected results — understanding Git's internals is the difference between recovering confidently and panicking. More importantly, once you understand the object model, everything Git does makes intuitive sense.

Git is fundamentally a content-addressable filesystem with a version control interface built on top. At its core, Git stores data as objects in a key-value store. The key is a SHA-1 hash of the content. The value is the content itself. Everything in Git — every file, every directory, every commit, every tag — is one of four types of objects stored in the `.git/objects` directory.

Blob objects store file content. When you add a file to Git, Git takes the file's content, computes its SHA-1 hash, and stores the content under that hash. Critically, Git stores content not filenames. Two different files with identical content share a single blob. If you rename a file without changing its content, no new blob is created. The blob is purely the raw file bytes.

Tree objects represent directories. A tree object contains a list of entries, each being either a blob reference (with a filename and permissions) or another tree reference (a subdirectory). A tree is essentially a directory snapshot — it records what files (blobs) and subdirectories (trees) existed and what their names and permissions were at a point in time.

Commit objects are what you create when you run `git commit`. A commit object contains: a reference to a tree object (the root directory snapshot at the time of commit), references to parent commit objects (zero parents for the initial commit, one for a normal commit, two for a merge commit), the author name and email, the committer name and email, timestamps, and the commit message. The commit points to the complete state of the repository at that moment.

Tag objects are annotated tags. They contain a reference to another object (usually a commit), a tag name, tagger identity, timestamp, and message. Lightweight tags (created with `git tag v1.0`) are just references, not objects. Annotated tags (`git tag -a v1.0 -m "Release"`) are full objects with metadata.

The SHA-1 hash of each object is determined by its content. If you create the exact same commit twice on different machines, you get the same SHA-1. If any byte changes, the hash changes. This is what makes Git's integrity guarantee work — you can verify any object by recomputing its hash.

References are how Git gives human-readable names to SHA-1 hashes. A branch like `main` is just a file in `.git/refs/heads/main` containing the SHA-1 of the commit it points to. When you commit on `main`, Git updates that file to point to the new commit. HEAD is a special reference that usually points to a branch (meaning HEAD points to the commit that branch points to). When you're in detached HEAD state, HEAD points directly to a commit SHA rather than to a branch.

Understanding this object model explains things that otherwise seem magical. Why is Git fast at branching? Because creating a branch just creates a new file containing a SHA-1 — it doesn't copy anything. Why can't you change a commit's content without changing its SHA? Because the SHA is derived from the content. Why does git rebase create new commits rather than moving existing ones? Because changing a commit's parent would change its SHA, making it a different object. The original commits still exist in the object store, unreachable until garbage collected.

To explore Git internals hands-on, `git cat-file -t <sha>` shows the type of any object, `git cat-file -p <sha>` shows its content, and `git log --graph --oneline` shows the commit graph structure.

02 Branching Strategies

How you organize branches in a repository is a strategic decision that affects how your team collaborates, how frequently you integrate code, how you manage releases, and how much overhead your development process has. Different branching strategies make different tradeoffs.

Git Flow was one of the first formalized branching strategies, introduced by Vincent Driessen in 2010. It has specific long-lived branches with specific purposes. `main` always contains production-ready code. `develop` is the integration branch where features are merged before release. Feature branches are created from `develop`, developed in isolation, then merged back to `develop`. Release branches are cut from `develop` when you're ready to release — only bug fixes go into the release branch. When the release is ready, it's merged to both `main` and `develop`. Hotfix branches are created from `main` to fix critical production bugs and are merged back to both `main` and `develop`.

Git Flow made sense when software was released in discrete versions with long release cycles. It's heavily used in mobile app development, packaged software, and any context where you ship distinct versions and must maintain multiple versions simultaneously. The downside is significant complexity — multiple long-lived branches create merge conflicts, require discipline to maintain, and create distance between developers' work and the main branch. For teams doing continuous delivery, Git Flow creates unnecessary friction.

Trunk-Based Development (TBD) is at the opposite extreme. There is exactly one long-lived branch — `main` (the trunk). Developers either commit directly to main (for small teams) or use very short-lived feature branches (hours to a couple of days, not weeks). Everyone integrates into main constantly. Releases are made from main. Feature flags control what's visible to users, not branches. There are no release branches, no develop branches, no hotfix branches.

TBD is the branching strategy used by Google, Facebook, and most high-functioning engineering organizations. It eliminates merge hell by ensuring code is integrated so frequently that conflicts are small and caught early. It requires strong CI/CD infrastructure (because main must always be deployable), good test coverage, and feature flags for incomplete features. The overhead is low but the quality bar for what goes into main is high.

GitHub Flow sits between Git Flow and Trunk-Based Development. There's one long-lived branch (`main`) and short-lived feature branches. Feature branches are created from main, developed, then merged back to main via pull request. Main is always deployable — you deploy from main. There are no develop branches or release branches. The simplicity makes it accessible to most teams, and the PR-based workflow enables code review. It works well for teams doing continuous delivery but doesn't handle the "maintain multiple released versions simultaneously" use case well.

Choosing a strategy — the right strategy depends on your release model. If you ship SaaS software with continuous deployment and one production version, Trunk-Based Development or GitHub Flow are appropriate. If you ship packaged software or mobile apps with versioned releases that require long-term support, Git Flow provides the structure you need. For most modern teams, GitHub Flow with short-lived branches and strong CI/CD is the right balance of simplicity and discipline.

For ML systems specifically, Trunk-Based Development works well for application and infrastructure code. Model development may use feature branches longer because experiments take time, but the principle of integrating frequently and using feature flags for incomplete features still applies.

03 Rebase vs Merge

Rebase and merge are both ways to integrate changes from one branch into another, but they produce fundamentally different histories and have different implications for collaboration. Understanding when to use each is one of the marks of Git maturity.

Merge combines two branches by creating a new merge commit that has two parents — one from each branch. The history shows exactly what happened: branch A diverged from main at point X, evolved independently, and was merged back at point Y. The original branch commits remain in the history unchanged.

If `feature-branch` was created from `main` at commit C3, and `main` has since advanced to C5 while `feature-branch` has commits C4' and C5':

diagram

4 lines

```
main:          C1 - C2 - C3 - C4 - C5
                  \
feature-branch: C4' - C5'
```

After merge:

diagram

4 lines

```
main: C1 - C2 - C3 - C4 - C5 - M1
                  \      /
feature-branch:    C4' - C5'
```

M1 is the merge commit with two parents (C5 and C5'). The history is accurate but can become complex — many active branches produce a graph that looks like a subway map and is hard to read linearly.

Rebase rewrites the feature branch commits to appear as if they were created on top of the current main, rather than at the point where the branch originally diverged. Rebase takes each commit on the feature branch, replays it on top of the target branch, and creates new commits with the same changes but different parent commits (and therefore different SHA-1s).

After rebasing `feature-branch` onto `main`:

diagram

4 lines

```
main:          C1 - C2 - C3 - C4 - C5
               |
feature-branch (rebased): C4'' - C5''
```

C4'' and C5'' have the same changes as C4' and C5' but are new commits (different SHAs) based on C5. Now you can fast-forward merge — since feature-branch is ahead of main in a straight line, merging just moves the main pointer forward without creating a merge commit:

diagram

2 lines

```
main: C1 - C2 - C3 - C4 - C5 - C4'' - C5''
```

The history is linear and clean. Reading git log is like reading a story without branching detours.

The golden rule of rebase — never rebase commits that have been pushed to a shared remote branch.

Rebase creates new commits (new SHAs) and abandons the old ones. If other developers have pulled your old commits and you rebase and force-push, their local repositories diverge from the remote in confusing ways. Rebase is safe for local work before sharing and for rebasing your local feature branch onto the latest main before creating a PR.

When to use merge: preserving exact history of how work happened, merging to shared long-lived branches (never rebase onto shared branches), and when you want an explicit record of when features were integrated.

When to use rebase: cleaning up local commits before pushing, integrating main's latest changes into your feature branch to resolve conflicts before merging, and producing a clean linear history in your repository.

Merge strategies — `git merge --no-ff` forces a merge commit even when fast-forward is possible, preserving the existence of the branch in history. `git merge --squash` squashes all changes from the feature branch into the working tree without committing, letting you create a single clean commit incorporating all the branch's changes. Squash merging is popular on GitHub for keeping main history clean while feature branches can have messy work-in-progress commits.

04 Interactive Rebase & History Rewriting

Interactive rebase (`git rebase -i`) is one of Git's most powerful features. It lets you rewrite the history of a series of commits — reordering them, combining them, splitting them, editing messages, removing them entirely — before sharing them with others. This is how you turn a messy series of work-in-progress commits into a clean, coherent history that tells a clear story.

To start an interactive rebase on the last 5 commits:

bash

2 lines

```
git rebase -i HEAD~5
```

Git opens an editor showing the last 5 commits in order (oldest first, opposite of `git log`):

```
pick a1b2c3d Fix model loading race condition
pick d4e5f6g Add feature store integration
pick h7i8j9k WIP: feature store
pick l0m1n2o Fix typo in config
pick p3q4r5s Add tests for feature store
```

You change the verb before each commit to control what happens to it:

pick — keep the commit as-is, unchanged.

reword — keep the commit's changes but edit the commit message. Git pauses the rebase and opens an editor for you to write a new message.

edit — keep the commit but pause the rebase at this point, letting you amend the commit (add files, change content) before continuing. Used for splitting a commit into multiple commits.

squash — combine this commit with the previous commit. Both commits' messages are combined and you edit the resulting message.

fixup — like squash, but discards this commit's message entirely, keeping only the previous commit's message. Used for commits like "fix typo" that don't deserve their own entry in history.

drop — delete this commit entirely and its changes.

reorder — just move the lines in the editor to change the order commits are applied.

In the example above, a clean history would squash the WIP commit and the typo fix into meaningful commits:

```
pick a1b2c3d Fix model loading race condition
pick d4e5f6g Add feature store integration
squash h7i8j9k WIP: feature store
pick l0m1n2o Fix typo in config    # could fixup into p3q4r5s
fixup p3q4r5s Add tests for feature store
```

Wait — "Fix typo in config" should be combined with the tests commit, so moving it:

```
pick a1b2c3d Fix model loading race condition
pick d4e5f6g Add feature store integration
squash h7i8j9k WIP: feature store
reword p3q4r5s Add tests for feature store
fixup l0m1n2o Fix typo in config
```

The result is three clean commits: the race condition fix, the feature store integration with all WIP work combined, and tests with the typo fix incorporated.

Splitting a commit — if a single commit does too many things, use `edit` to pause at that commit, `git reset HEAD~` to unstage all the commit's changes (but keep them in working tree), then stage and commit them in logical groups. Then `git rebase --continue` to proceed.

Force pushing after history rewriting — after interactive rebase, your local branch has different commits (different SHAs) than the remote. You must force push: `git push --force-with-lease origin feature-branch`. The `--force-with-lease` variant is safer than `--force` — it checks that nobody else has pushed to the branch since you last fetched. If someone has, the push is rejected instead of overwriting their work.

When to rewrite history — only before sharing. Once commits are pushed to a shared branch that others have pulled, rewriting creates confusion. For personal feature branches where you're the only one working, clean up aggressively before creating the PR. The person reviewing your PR shouldn't need to wade through "WIP", "fix", "another fix", "typo", "actually fix" commits.

05 Cherry-Pick, Revert & Reset Strategies

These three commands give you surgical control over Git history and the working state, each serving a distinct purpose that's important to understand clearly.

Cherry-pick applies the changes introduced by a specific commit onto the current branch. It creates a new commit with the same changes but a different SHA (different parent, possibly different timestamp). Cherry-pick is useful when you want to apply a specific fix from one branch to another without merging the entire branch.

bash

2 lines

```
git cherry-pick a1b2c3d
```

This takes whatever changes commit `a1b2c3d` introduced and applies them to your current branch. If you're on a release branch and need a specific bug fix from main, cherry-pick that fix's commit onto the release branch.

Cherry-pick becomes complicated when the commit depends on changes that don't exist on the target branch — the patch may not apply cleanly and conflicts arise. Also, if you cherry-pick many commits from a branch and later merge that branch, Git sees duplicate changes (same diff applied twice) and creates messy conflicts. Cherry-pick is best used sparingly for isolated, independent changes.

Revert creates a new commit that undoes the changes of a specific previous commit. Unlike reset (which actually removes commits), revert adds a new commit to the history. This is the safe way to undo a change that has already been pushed to a shared branch, because it preserves the history.

bash

2 lines

```
git revert a1b2c3d
```

This creates a new commit applying the inverse of what `a1b2c3d` did — if that commit added a line, the revert removes it. The original commit remains in history. If you deployed a bad model configuration and need to undo it immediately, revert the commit and push — this creates a clean, auditable record that the change was made and then undone, rather than erasing it from history.

Reverting merge commits requires the `-m` flag specifying which parent to revert to: `git revert -m 1 <merge-commit-sha>`. The `1` specifies the first parent (the branch that was merged into), effectively undoing the merge.

Reset moves the current branch pointer and optionally changes the working tree and staging area. It has three modes that are critically different:

`git reset --soft HEAD~3` — moves the branch pointer back 3 commits but leaves the changes from those commits staged in the index. The commits disappear but their changes are ready to be recommitted. Use this to squash recent commits into one: reset soft to the commit before the ones you want to squash, then `git commit` to create one commit with all the changes.

`git reset --mixed HEAD~3` — moves the branch pointer back 3 commits and unstages the changes (but leaves them in the working tree). This is the default mode. The changes are still there but unstaged — you can selectively stage and recommit them. Use this when you want to restructure how changes are split across commits.

`git reset --hard HEAD~3` — moves the branch pointer back 3 commits and throws away all changes from those commits. The working tree and staging area are reset to the state of the target commit. This is destructive — the changes are gone (though recoverable via `git reflog` for a while). Use this when you genuinely want to abandon work.

Recovery with reflog — Git maintains a reference log (`git reflog`) that records every time HEAD moves, even for operations like `reset --hard`. Each reflog entry has a SHA. If you accidentally reset `--hard` and lose work, `git reflog` shows you what HEAD pointed to before the reset, and you can `git checkout <sha>` or `git reset --hard <sha>` to recover. The reflog is local and not pushed to remotes, but it persists for 90 days by default.

06 Git Bisect for Debugging

Git bisect is a tool for finding which commit introduced a bug using binary search. If you have a repository with 1000 commits and you know the software worked at commit A but doesn't work at commit Z, git bisect finds the exact commit that broke it in about 10 steps ($\log_2(1000) \approx 10$) instead of manually checking hundreds of commits.

The process is straightforward. You tell Git where the problem was definitely not present (a good commit) and where it definitely exists (a bad commit), and Git automatically checks out commits in the middle, asking you to classify each one. Binary search halves the search space with each classification.

bash

22 lines

```
# Start bisecting
git bisect start

# Tell Git the current commit is bad
git bisect bad

# Tell Git where you know things worked
git bisect good v2.0.0
```



```
# Git checks out the midpoint commit
# Test your application...

# If this commit has the bug:
git bisect bad

# If this commit doesn't have the bug:
git bisect good

# Git keeps halving the range until it identifies the first bad commit
# When done:
git bisect reset
```

Automated bisect is where bisect becomes truly powerful. If you have a test or script that automatically determines whether a commit is good or bad, you can automate the entire process:

bash

5 lines

```
git bisect start
git bisect bad HEAD
git bisect good v2.0.0
git bisect run python test_model_accuracy.py
```

Git runs `test_model_accuracy.py` at each midpoint commit. The script should exit 0 if the commit is good, exit 1 if bad, and exit 125 to skip a commit (if the commit doesn't compile or can't be tested for unrelated reasons). Git runs this automatically until it finds the first bad commit, potentially finding it while you're in a meeting.

For ML systems, this is valuable for finding which code commit caused a model metric regression, which configuration change introduced a performance degradation, or which dependency update broke an inference path. The test script can be as sophisticated as needed — train a small model, run inference on a test set, check that accuracy is above a threshold.

After bisect identifies the culprit commit, you read that commit's diff with `git show <bad-commit-sha>` to understand exactly what change caused the issue.

07 Git Hooks

Git hooks are scripts that Git executes automatically before or after specific events — committing, pushing, receiving a push, merging. They're how you enforce standards, run checks, and automate processes that should happen at specific points in the Git workflow.

Hooks live in the `.git/hooks/` directory. Creating an executable file with the right name makes it a hook. The hooks are just scripts — they can be bash, Python, Ruby, or any other executable.

Client-side hooks run on the developer's machine:

`pre-commit` runs before a commit is created, after you run `git commit` but before the commit message editor opens. If this hook exits with a non-zero code, the commit is aborted. This is where you enforce code formatting (run black for Python, prettier for JavaScript), static analysis (run flake8, mypy), check for accidentally committed secrets (scan for API keys, passwords), and validate that test files exist for new features.

bash

18 lines

```
#!/bin/bash
# pre-commit hook example
echo "Running pre-commit checks..."

# Check for Python formatting
black --check src/ || { echo "Run 'black src/' to fix formatting"; exit 1; }

# Check for type errors
mypy src/ --ignore-missing-imports || exit 1

# Check for secrets accidentally committed
if grep -r "AWS_SECRET_ACCESS_KEY|password|api_key" --include="*.py" src/; then
    echo "Potential secret detected in code!"
    exit 1
fi

echo "Pre-commit checks passed"
```

`commit-msg` receives the commit message as a file argument and can validate or modify it. Use this to enforce commit message conventions — Conventional Commits format (`feat:` , `fix:` , `docs:`), Jira ticket references, minimum length requirements.

`pre-push` runs before git push sends commits to the remote. This is where you run the full test suite (too slow for pre-commit). Aborting a push that would break CI saves time and keeps the shared branch green.

Server-side hooks run on the Git server (GitHub, GitLab, or self-hosted Git):

`pre-receive` runs on the server when it receives a push, before accepting it. It can reject pushes that don't meet requirements — commits without GPG signatures, pushes to protected branches from unauthorized users, changes that would exceed file size limits. This enforces standards that can't be bypassed by modifying local hooks.

`post-receive` runs after the push is accepted and is typically used to trigger deployments, send notifications, or update CI systems. This was how simple CD pipelines worked before dedicated CI/CD tools became standard.

Managing hooks in teams — the `.git/hooks/` directory is not tracked by Git (it's inside `.git` , not in the working tree). This means hooks aren't shared automatically with team members. Solutions: store hook scripts in a tracked directory (like `.githubhooks/`) and set the hook directory with `git config core.hooksPath`

`.githubhooks` . Or use a hooks management tool like pre-commit (the Python tool), which defines hooks in `.pre-commit-config.yaml` that is tracked in the repository, automatically runs hooks in isolated environments, and manages hook versions consistently across all developers.

yaml

15 lines

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/psf/black
  rev: 23.9.1
  hooks:
  - id: black
- repo: https://github.com/PyCQA/flake8
  rev: 6.1.0
  hooks:
  - id: flake8
- repo: https://github.com/Yelp/detect-secrets
  rev: v1.4.0
  hooks:
  - id: detect-secrets
```

`pre-commit install` sets up the hooks for any developer who runs it.

08 Git Submodules & Monorepo Strategy

As organizations grow, they face a fundamental repository organization question: should all code live in one repository (monorepo), or should each project/service have its own repository (polyrepo)? And if multiple repositories exist, how do you handle shared code and dependencies?

Git Submodules are Git's built-in mechanism for embedding one repository inside another. A submodule is a reference to a specific commit in another repository. If your ML platform depends on a shared internal library, you can include that library's repository as a submodule:

bash

2 lines

```
git submodule add https://github.com/myorg/ml-utils.git libs/ml-utils
```

This creates a `.gitmodules` file recording the submodule's URL and path, and records the specific commit of ml-utils that the parent repository currently uses. When someone clones your repository, they need `git clone --recurse-submodules` to also pull the submodule content, or run `git submodule update --init` after a regular clone.

Updating a submodule means navigating into the submodule directory, pulling the new commits, navigating back to the parent repository, staging the submodule change (which updates the recorded commit pointer), and committing. The parent repository records a specific commit of the submodule, not a branch — you're pinning the exact version used.

Submodules solve a real problem but introduce real friction. The workflow is confusing for developers unfamiliar with them. Forgetting to initialize submodules after cloning produces mysterious missing-file errors. Updating submodules across many repositories is tedious. They work well for small organizations with a few clear shared libraries but don't scale to complex dependency graphs.

Monorepo strategy puts all code — every service, every library, every tool — in a single repository. Google, Meta, Microsoft, Twitter, and Uber all use monorepos. The advantages are significant: atomic cross-service changes (one PR changes the shared library and all services that use it simultaneously), consistent tooling and standards across all code, easy discovery of all code in the organization, simplified dependency management (no versioning of internal dependencies), and large-scale refactoring that spans multiple services.

The challenges are also real: the repository becomes enormous (Google's is over 80GB), standard Git operations that scan the entire repository become slow, CI/CD must be sophisticated enough to only run tests for what changed (running all tests in a monorepo on every commit is impractical), and access control is more complex when everything is in one place.

Monorepo tools address the scale challenges. Nx (primarily for JavaScript/TypeScript), Bazel (Google's build tool, language-agnostic), Turborepo, and Pants are build systems designed for monorepos. They understand the dependency graph of your codebase and can determine exactly which services are affected by a given change, running only the relevant tests. This is called affected-based CI and is what makes monorepo CI tractable at scale.

For ML systems, a practical approach is a small number of repositories organized by concern rather than one extreme or the other. Infrastructure code in one repo (Terraform, Kubernetes configs), ML platform code in another (the training and serving platform), individual ML project code in their own repos or as directories in a shared ML repo. Internal shared libraries (data utilities, model evaluation frameworks) can be managed as Python packages published to a private PyPI registry, which avoids the submodule complexity while solving the shared code problem.

09 Semantic Versioning

Semantic versioning (SemVer) is a versioning convention that communicates meaning about the nature of changes between releases through the version number itself. Instead of arbitrary version numbers, SemVer version numbers tell you whether a new release contains bug fixes, new features, or breaking changes.

A SemVer version has the format `MAJOR.MINOR.PATCH` — for example `2.4.1`. Each component has a specific meaning:

MAJOR version increments when you make incompatible API changes — changes that break existing users of your software. If you rename a function, remove a parameter, change a return type, or restructure an API in any way that requires callers to update their code, that's a major version bump. `1.5.3` → `2.0.0`.

MINOR version increments when you add new functionality in a backward-compatible manner. New functions, new parameters with defaults, new configuration options — existing users don't need to change anything, but new functionality is available. `2.4.1` → `2.5.0`.

PATCH version increments when you make backward-compatible bug fixes. No new features, no API changes — just fixes to existing behavior. `2.4.1` → `2.4.2`.

Additional labels are allowed for pre-release versions (`2.5.0-alpha.1` , `2.5.0-beta.3` , `2.5.0-rc.1`) and build metadata (`2.5.0+20231015`).

Why SemVer matters — when you specify dependencies using SemVer-aware version constraints, you can reason about what updates are safe. In Python with pip, `torch ≥ 2.0.0, < 3.0.0` means "any PyTorch version from 2.0.0 up to but not including 3.0.0 is compatible" — new minor and patch versions will be used automatically, but major version 3.0 (which might have breaking changes) won't be automatically adopted. This is the contract SemVer establishes between library maintainers and library users.

For ML systems, versioning carries additional meaning beyond the code API. Your model serving API might be versioned separately from the model itself. `GET /v1/predict` and `GET /v2/predict` can coexist, letting you introduce a new prediction format without immediately breaking all clients. Model weights themselves should be versioned — training a new model and serving it doesn't break the API, but it changes model behavior in ways that need version tracking for reproducibility.

10 Tagging & Release Management

Tags in Git are references to specific commits that, unlike branches, don't move when new commits are added. They're typically used to mark release points — this specific commit is version 2.4.1 of the software.

Lightweight tags are just a named pointer to a commit:

bash

3 lines

```
git tag v2.4.1
git tag v2.4.1 a1b2c3d # Tag a specific commit
```

Annotated tags are full objects with metadata — tagger identity, date, and message — and can be GPG-signed:

bash

3 lines

```
git tag -a v2.4.1 -m "Release 2.4.1: fixes model loading race condition"
git tag -a v2.4.1 -s # -s for GPG signing
```

Annotated tags are preferred for releases because they include the who and why of the release, and they can be verified if signed.

Tags are not pushed automatically with `git push`. Push tags explicitly:

bash

3 lines

```
git push origin v2.4.1 # Push a specific tag
git push origin --tags # Push all tags
```

Release management workflow — a mature release process looks like this. Development happens on main (or feature branches merged to main). When ready to release, a release branch is cut from main (`release/2.4.0`). The release branch gets final testing, bug fixes cherry-picked from main if needed, and

version number updates in code files. When the release is ready, the release commit is tagged (`v2.4.0`). If using a release branch, it's merged to main. The tag is pushed to trigger the release pipeline.

Automated release notes — tools like `git-chglog` and `release-please` automatically generate changelogs and release notes from commit messages, provided you follow Conventional Commits format (`feat:` , `fix:` , `chore:` , `docs:` prefixes). `release-please` runs as a CI job, creates a "release PR" that bumps version numbers and generates a changelog, and when merged, creates the tag and GitHub release automatically.

GitHub Releases — creating a GitHub Release (distinct from a Git tag) provides a user-facing release page with assets (binary downloads, source archives), rendered release notes, and integration with tools that watch for releases.

11 Secure Git Workflows

Git security is often overlooked but important. Code repositories contain your entire history, potentially including accidentally committed secrets, and the ability to push code is the ability to affect what runs in production.

Commit signing with GPG — by default, anyone can create a commit claiming any author email. Someone could create a commit claiming to be authored by your CEO. GPG commit signing cryptographically proves that the person who made the commit actually has the private key associated with the author's identity. GitHub shows a "Verified" badge on signed commits. Requiring signed commits on protected branches ensures all commits can be cryptographically traced to a specific developer's key.

bash

6 lines

```
# Set your signing key
git config --global user.signingkey YOUR_GPG_KEY_ID

# Sign commits by default
git config --global commit.gpgsign true
```

Protected branches — in GitHub, GitLab, and similar platforms, branch protection rules prevent force pushes to specific branches, require pull request reviews before merging, require CI checks to pass before merging, and require signed commits. Main and release branches should always be protected. Nobody, including administrators, should be able to push directly to main in a production repository.

Secret detection — accidentally committed secrets (API keys, database passwords, cloud credentials embedded in config files or test data) are a leading cause of security incidents. Pre-commit hooks with `detect-secrets` or `truffleHog` scan staged files for patterns matching common secret formats before each commit. Server-side `pre-receive` hooks (or GitHub's secret scanning feature) scan pushes server-side as a second line of defense. Neither is perfect — some secrets don't match known patterns — but together they catch the majority of accidental commits.

When a secret is committed — if a secret is committed, treat the secret as fully compromised immediately, even if you delete it in the next commit. Git history is preserved and the secret is visible in the old commit. You must rotate the credential immediately (generate a new API key, change the password), then optionally

use `git filter-branch` or BFG Repo Cleaner to remove the secret from history and force-push all branches. This is disruptive and not always worth the effort once the credential is rotated, but may be required for compliance.

Access control — follow least privilege for repository access. Developers get write access to feature branches but not direct push to main. Automated systems (CI/CD) get the minimum access needed — a deploy pipeline doesn't need write access to the repository. Use deploy keys (single-repository SSH keys) for CI/CD rather than personal access tokens that could access all repositories. Regularly audit who has access and revoke access promptly when someone leaves the organization.

ADVANCED CI/CD ENGINEERING

12 CI/CD Architecture & Pipeline Design

CI/CD (Continuous Integration and Continuous Delivery/Deployment) is the practice of automating the path from code change to production deployment. CI catches problems early by building and testing every change. CD automates the delivery of validated changes to production. Together, they make deploying software fast, reliable, and low-risk.

Before designing pipelines, understand the vocabulary. **Continuous Integration** means every code change is automatically built, tested, and validated — catching integration problems early when they're cheap to fix.

Continuous Delivery means the software is always in a state where it could be deployed to production — the pipeline validates every change to the point of production readiness, but actual deployment is a manual decision. **Continuous Deployment** takes this further — every change that passes all automated validation is automatically deployed to production without human intervention.

Most mature engineering organizations practice Continuous Delivery at minimum and Continuous Deployment for non-critical services.

Pipeline architecture principles that separate good pipelines from bad ones:

Fast feedback first — the most common reason developers disable or ignore CI is because it takes too long. A 45-minute pipeline means developers switch to something else, context-switch back when it finishes, and fix problems with cold context. Structure your pipeline so the fastest, most common failure modes run first. Linting and type checking in seconds. Unit tests in a few minutes. Integration tests after those pass. Full end-to-end tests last. Developers get feedback on the most common problems within 2-3 minutes.

Fail fast — don't continue running expensive stages if cheap stages have already found a problem. If linting fails, don't start building Docker images. If unit tests fail, don't run integration tests. Exit the pipeline at the earliest possible point.

Reproducibility — pipelines should produce the same result every time they run on the same input. This means pinning tool versions, using dependency lock files, using hermetic build environments. A pipeline that fails intermittently because it pulls `latest` of a tool that changed is harder to debug than application bugs.

Parallelism — many checks are independent and can run simultaneously. Lint Python code, lint YAML, check Terraform format, and check Dockerfile best practices can all run in parallel. Running them sequentially wastes time.

Visibility — every pipeline run should be observable. Clear stage names, structured logs, artifact storage with retention, metrics on pipeline duration and failure rate. When a pipeline fails at 2am, the on-call engineer shouldn't have to decode cryptic output to understand what happened.

For ML systems, CI/CD has additional stages beyond standard software pipelines: model training runs (at least on a sample dataset to validate training code), model evaluation (does the trained model meet minimum metric thresholds?), model registration (pushing validated models to the model registry), and model deployment stages for each environment.

13 Pipeline as Code

Pipeline as Code means your CI/CD pipeline definition is stored in version control as code, not configured through a web UI. This seems obvious but represents a significant shift from how CI was done historically — Jenkins was configured through GUI, pipeline definitions lived in the CI server's database and couldn't be version-controlled, reviewed, or easily replicated.

Modern CI/CD systems (GitHub Actions, GitLab CI, CircleCI, Tekton, Argo Workflows) define pipelines as YAML files stored in the repository. This gives you: version history of pipeline changes — who changed what when, code review for pipeline changes through PR review, the ability to run different pipeline versions for different branches, easy replication across repositories, and local development and testing of pipeline logic.

A GitHub Actions workflow file:

yaml

37 lines

```
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with:
          python-version: "3.11"
      - run: pip install black flake8 mypy
      - run: black --check src/
      - run: flake8 src/
      - run: mypy src/

  test:
    needs: lint
```



```

runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v4
  with:
    python-version: "3.11"
- run: pip install -r requirements.txt
- run: pytest tests/ --junitxml=test-results.xml
- uses: actions/upload-artifact@v3
  with:
    name: test-results
    path: test-results.xml

```

The **needs** keyword creates dependencies between jobs — `test` only runs after `lint` passes. This implements the fast-fail principle at the job level.

Pipeline as Code best practices: store pipeline files in a discoverable location (`.github/workflows/` for GitHub Actions, `.gitlab-ci.yml` for GitLab CI), name pipelines and jobs descriptively, comment complex logic, treat pipeline code with the same review rigor as application code, and don't put sensitive logic (hardcoded credentials, internal hostnames) in pipeline files — use secrets and environment variables.

14 Multi-Stage Pipeline Design

A multi-stage pipeline organizes the build process into distinct stages that run in sequence, with each stage building on the success of the previous. This structure makes pipelines readable, enables selective re-running of failed stages, and enforces the principle that later, more expensive stages only run when earlier, cheaper stages pass.

A mature multi-stage ML CI/CD pipeline looks like this:

Stage 1: Validate (30-60 seconds) — fast checks that catch the most common problems immediately. Code formatting (black, prettier), style (flake8, eslint), type checking (mypy), YAML linting, Dockerfile linting (hadolint), security static analysis (bandit for Python). Every developer gets this feedback within a minute.

Stage 2: Test (2-10 minutes) — unit tests for all application code, property-based tests for data transformations, mocked tests for ML pipeline components. Tests here must be hermetic — no network calls, no real databases, everything mocked.

Stage 3: Build (5-15 minutes) — build Docker images, compile any native extensions, generate documentation. Tag images with the commit SHA. Scan images for vulnerabilities. Sign images.

Stage 4: Integration Test (10-30 minutes) — spin up real services using Docker Compose or ephemeral Kubernetes namespaces. Run tests against real databases, real message queues, real feature stores. Validate that the application works with real infrastructure.

Stage 5: ML Validation (15-60 minutes) — run a training job on a sample dataset. Evaluate the trained model against baseline metrics. Validate data schemas and feature distributions. Register the model in the staging model registry if metrics pass.

Stage 6: Deploy to Staging (5-10 minutes) — deploy the built images and validated models to the staging environment. Run smoke tests against staging. Validate critical paths are functional.

Stage 7: Deploy to Production — either automated (Continuous Deployment) with automated smoke tests and monitoring, or requires a manual approval gate before proceeding. Canary deployment with automated metric checks before full rollout.

Each stage only runs if all previous stages passed. A failure in Stage 2 (unit tests) stops the pipeline — you don't build images or run integration tests for code that has failing unit tests. This saves significant compute time and keeps feedback loops tight.

15 Matrix Builds & Parallel Execution

Matrix builds run the same job multiple times with different configurations simultaneously. Instead of testing against only one Python version or one OS, you test against all supported combinations in parallel, getting comprehensive coverage in the same time as a single configuration test.

GitHub Actions matrix strategy:

yaml

20 lines

```
jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        python-version: ["3.9", "3.10", "3.11"]
        exclude:
          - os: windows-latest
            python-version: "3.9" # Skip this specific combination
      fail-fast: false # Don't cancel other matrix jobs if one fails

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with:
          python-version: ${ matrix.python-version }
      - run: pip install -r requirements.txt
      - run: pytest tests/
```

This creates 8 jobs (3 OS × 3 Python versions, minus the 1 excluded combination) that run simultaneously. The whole matrix completes in the time of a single test run rather than 8 times as long.

For ML systems, matrices are useful for testing model serving on different hardware configurations, testing data pipeline code against different database versions, validating that training code works with different framework versions (torch 2.0 and torch 2.1), and testing infrastructure code against different Kubernetes versions.

Parallel execution without matrices — jobs that are independent should always run in parallel. Linting Python code and linting Terraform code are completely independent — run them simultaneously. Building multiple Docker images for different services can run in parallel. GitHub Actions, GitLab CI, and most CI systems run

independent jobs in parallel by default when multiple runners are available.

Controlling parallelism — too much parallelism can overwhelm dependent services (like a test database) with concurrent connections. The `concurrency` setting in GitHub Actions limits how many runs of a workflow can run simultaneously, and the `max-parallel` in matrix strategy limits how many matrix jobs run at once.

16 Reusable Workflows & Composite Actions

As you build pipelines for multiple repositories or multiple services within a monorepo, you'll notice significant duplication. Every repository has the same Python setup steps, the same Docker build and push steps, the same deployment steps. Copy-pasting pipeline code across repositories is the pipeline equivalent of copy-pasting application code — it works until you need to change something, at which point you need to update dozens of pipeline files.

Reusable workflows and composite actions are GitHub Actions' mechanisms for DRY pipeline code.

Composite Actions package a series of steps into a reusable action that can be called from any workflow. You define the action in a repository (can be the same repository or a shared actions repository), and call it with parameters.

Creating a composite action at `.github/actions/build-and-push/action.yml`:

yaml

47 lines

```
name: Build and Push Docker Image
description: Builds a Docker image and pushes to ECR

inputs:
  image-name:
    required: true
    description: Name of the Docker image
  dockerfile:
    required: false
    default: Dockerfile
    description: Path to Dockerfile
  aws-region:
    required: true
    description: AWS region for ECR

outputs:
  image-uri:
    description: Full ECR image URI with digest
    value: ${ steps.push.outputs.image-uri }

runs:
  using: composite
  steps:
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
```

```

with:
  role-to-assume: ${ env.ECR_PUSH_ROLE }
  aws-region: ${ inputs.aws-region }

- name: Login to ECR
  uses: aws-actions/amazon-ecr-login@v2

- name: Build image
  shell: bash
  run: |
    docker build -f ${ inputs.dockerfile } \
      -t ${ inputs.image-name }:${ github.sha } .

- name: Push and get digest
  id: push
  shell: bash
  run: |
    docker push ${ inputs.image-name }:${ github.sha }
    DIGEST=$(docker inspect --format='{{index .RepoDigests 0}}' \
      ${ inputs.image-name }:${ github.sha })
    echo "image-uri=$DIGEST" >> $GITHUB_OUTPUT

```

Using this action:

yaml

5 lines

```

- uses: ../github/actions/build-and-push
  with:
    image-name: myregistry/model-server
    aws-region: us-east-1

```

Reusable Workflows are entire workflow files that can be called from other workflows, enabling sharing of complete pipeline stages across repositories:

yaml

24 lines

```

# .github/workflows/reusable-deploy.yml
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
      image-tag:
        required: true
        type: string
    secrets:

```

```

    kube-config:
      required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${ inputs.environment }
    steps:
      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/model-server \
            model-server=myregistry/model-server:${ inputs.image-tag }

```

Calling this from another workflow:

yaml

8 lines

```

deploy-staging:
  uses: myorg/.github/.github/workflows/reusable-deploy.yml@main
  with:
    environment: staging
    image-tag: ${ github.sha }
  secrets:
    kube-config: ${ secrets.STAGING_KUBE_CONFIG }

```

Storing reusable workflows and composite actions in a dedicated `.github` repository in your organization makes them available to all repositories. Changes to shared actions propagate to all consumers when they update their references.

17 Environment-Based Deployments

Environments in CI/CD represent the stages infrastructure passes through from development to production — typically development, staging, and production. Each environment has different infrastructure, different secrets, different approval requirements, and different deployment strategies.

GitHub Actions has native environment support with protection rules:

```
deploy-production:
  needs: [deploy-staging, integration-tests]
  runs-on: ubuntu-latest
  environment:
    name: production
    url: https://api.company.com
  steps:
  - name: Deploy to production
    run: kubectl apply -f k8s/production/
```

You configure the `production` environment in GitHub's Settings → Environments with: required reviewers (specific people who must approve before the job runs), wait timer (mandatory delay between staging deployment and production deployment), deployment branch rules (only the main branch can deploy to production), and environment secrets (secrets specific to this environment, not shared with other environments).

When a workflow reaches a job with an environment that has required reviewers, the workflow pauses. The designated reviewers get a notification, review what's being deployed (the PR it came from, the test results), and click Approve or Reject. This human gate is critical for production deployments.

Environment-specific configuration — each environment needs different secrets (different database URLs, different API keys, different cloud credentials). GitHub Environments let you store secrets scoped to specific environments. A `DATABASE_URL` secret in the `staging` environment contains the staging database URL, while the same-named secret in `production` contains the production URL. The workflow code is identical — the environment determines which secret value is used.

Deployment tracking — when a workflow deploys using an environment, GitHub records the deployment event, links it to the commit and PR, and shows it in the repository's "Deployments" view. This gives you a complete history of what was deployed to each environment and when, directly connected to the code changes.

18 Secrets Management in CI/CD

CI/CD pipelines need credentials to do their work — cloud provider credentials to deploy infrastructure, registry credentials to push images, Kubernetes credentials to deploy services, database passwords to run integration tests, API tokens to communicate with external services. Managing these credentials securely is one of the most critical aspects of CI/CD security.

Never store secrets in pipeline files — this seems obvious but the temptation to hardcode a credential "just temporarily" is constant. Pipeline files are code that goes into version control. Secrets in version control are exposed to everyone with repository access and are preserved permanently in history even if removed. Always use the secret storage mechanism provided by your CI/CD platform.

CI/CD platform secrets — GitHub Actions Secrets, GitLab CI Variables, CircleCI Contexts all provide encrypted storage for secrets that are injected into pipeline jobs as environment variables. They're not visible in logs, not accessible in pull requests from forks, and can be scoped to specific environments.

```
steps:
- name: Deploy to EKS
  env:
    AWS_ROLE_ARN: ${ secrets.PROD_AWS_ROLE_ARN }
  run: |
    aws eks update-kubeconfig --name production-cluster
    kubectl apply -f k8s/
```

OIDC federation — eliminating static credentials — the most secure approach for cloud deployments is eliminating long-lived credentials entirely using OpenID Connect (OIDC) federation. GitHub Actions, GitLab CI, and CircleCI all support OIDC. Your cloud provider (AWS, GCP, Azure) trusts the CI system's OIDC issuer. The CI job requests a short-lived token from the OIDC issuer, presents it to the cloud provider, and receives temporary cloud credentials valid only for the duration of the job. There are no long-lived access keys stored anywhere — no credential to leak, no credential to rotate.

```
- name: Configure AWS credentials via OIDC
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789:role/GitHubActionsDeployRole
    aws-region: us-east-1
```

AWS verifies that this job is running in the specific repository and specific branch configured in the IAM role's trust policy. If the job is running in a different repository (indicating a compromise), authentication fails.

Vault integration — for organizations using HashiCorp Vault, CI jobs authenticate to Vault using their OIDC identity and retrieve secrets dynamically. Secrets are never stored in the CI platform — they're fetched from the authoritative secret store at the time the job needs them.

Secret rotation — CI secrets should be rotated regularly. With OIDC-based authentication, there's nothing to rotate — credentials are ephemeral by design. For static secrets that must be stored, establish a rotation schedule and automate rotation where possible.

19 Dependency Caching Strategies

Package installation is often the slowest part of a CI pipeline. Installing PyTorch with its CUDA dependencies might take 5-10 minutes. Running this on every pipeline run when the dependencies haven't changed is pure waste.

Dependency caching stores the installed packages from one pipeline run and restores them in future runs. If the cache is valid (dependencies haven't changed), the cache restore takes seconds instead of minutes.

The key to effective caching is the cache key — the value that determines whether a cache hit or miss occurs. The cache key must uniquely identify the combination of dependencies. For Python, the right key is a hash of the requirements file — if `requirements.txt` changes, the hash changes, the cache misses, and

packages are freshly installed. If requirements haven't changed, the hash matches, the cache hits, and packages are restored instantly.

yaml

13 lines

```
- name: Cache Python dependencies
  uses: actions/cache@v3
  id: pip-cache
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt', 'requirements-dev.txt') }}
    restore-keys: |
      ${{ runner.os }}-pip-

- name: Install dependencies
  if: steps.pip-cache.outputs.cache-hit != 'true'
  run: pip install -r requirements.txt -r requirements-dev.txt
```

The `restore-keys` provides fallback cache keys — if no exact match exists, restore the most recent cache with the prefix. This gives you a partial cache (most packages installed, perhaps missing the newest additions) which is still faster than installing from scratch.

Docker layer caching — Docker builds can also use caching in CI. BuildKit supports using a registry as a cache backend:

bash

6 lines

```
docker buildx build \
  --cache-from type=registry,ref=myregistry/model-server:buildcache \
  --cache-to type=registry,ref=myregistry/model-server:buildcache,mode=max \
  -t myregistry/model-server:$SHA \
  .
```

Unchanged layers are pulled from the registry cache and not rebuilt, dramatically accelerating image builds that have many layers.

Cache invalidation — the hardest problem in CI caching is knowing when caches are stale. Hash-based keys handle this for dependency files. But for caches keyed on other things (branch name, date), you may occasionally need to manually invalidate caches by changing the key prefix. Some teams append a cache version number to keys and increment it when they need to force invalidation across all branches.

20 Artifact Management

Artifacts are the outputs of CI/CD pipeline stages that need to be stored and potentially passed between stages or retained for later reference. Test reports, compiled binaries, Docker images, trained models, Terraform plans, coverage reports — all of these are artifacts.

Artifact storage in CI platforms — GitHub Actions, GitLab CI, and CircleCI all have built-in artifact storage. You upload artifacts at the end of a job, and they're retained for a configurable period (7-90 days typically). Subsequent jobs in the same pipeline can download artifacts from previous jobs, enabling inter-job communication without a shared filesystem.

yaml

24 lines

```
build:
  runs-on: ubuntu-latest
  steps:
    - name: Build application
      run: python -m build
    - name: Upload build artifacts
      uses: actions/upload-artifact@v3
      with:
        name: dist-packages
        path: dist/
        retention-days: 7

deploy:
  needs: build
  runs-on: ubuntu-latest
  steps:
    - name: Download build artifacts
      uses: actions/download-artifact@v3
      with:
        name: dist-packages
        path: dist/
    - name: Deploy
      run: ./deploy.sh dist/
```

Artifact naming and versioning — artifacts should be named descriptively and consistently. Include the pipeline run ID, commit SHA, or version number in artifact names to ensure uniqueness and traceability.

`model-server-v2.4.1-linux-amd64.tar.gz` is better than `build.tar.gz`.

External artifact registries — for important artifacts with long retention requirements, store them in purpose-built registries rather than relying on CI platform artifact storage which has limited retention. Docker images go to container registries (ECR, GCR). Python packages go to private PyPI (Artifactory, AWS CodeArtifact). Helm charts go to chart repositories. Terraform modules go to the Terraform registry or an S3 bucket. ML models go to the model registry (MLflow, Weights & Biases).

Artifact provenance — knowing where an artifact came from is essential for security and debugging. Your artifacts should be traceable back to the exact source code commit and pipeline run that produced them. Attaching SBOM, SLSA provenance attestations, and build logs to artifacts creates a complete chain of custody from source code to deployed artifact.

21 Docker Build & Push Automation

Automating Docker image builds and pushes is a foundational CI/CD capability for any containerized system. Every code change that affects deployed services should trigger an automated build, producing a versioned, scanned, signed image pushed to the registry.

A complete Docker build and push workflow:

yaml

75 lines

```
name: Build and Push

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  id-token: write # For OIDC
  contents: read
  security-events: write # For Sarif upload

jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.ECR_PUSH_ROLE }
          aws-region: us-east-1

      - name: Login to ECR
        id: ecr-login
        uses: aws-actions/amazon-ecr-login@v2

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${ steps.ecr-login.outputs.registry }/model-server
          tags: |
            type=sha,prefix=sha-
            type=semver,pattern={{version}}
            type=ref,event=branch

      - name: Build and push
        id: build
        uses: docker/build-push-action@v5
```

```

with:
  context: .
  push: ${ github.event_name != 'pull_request' }
  tags: ${ steps.meta.outputs.tags }
  labels: ${ steps.meta.outputs.labels }
  cache-from: type=gha
  cache-to: type=gha,mode=max
  provenance: true
  sbom: true

- name: Scan image
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ steps.ecr-login.outputs.registry }/model-server:sha-${ github.sha }
    format: sarif
    output: trivy-results.sarif
    severity: CRITICAL,HIGH
    exit-code: 1

- name: Upload scan results
  uses: github/codeql-action/upload-sarif@v2
  with:
    sarif_file: trivy-results.sarif

- name: Sign image
  if: github.event_name != 'pull_request'
  run: |
    cosign sign --yes \
      ${ steps.ecr-login.outputs.registry }/model-server@${ steps.build.outputs.digest }

```

This workflow builds with BuildKit layer caching, pushes with multiple tags, generates SBOM and provenance attestations, scans for vulnerabilities, uploads scan results to GitHub Security, and signs the image — all in a single automated pipeline.

On PR — the image is built but not pushed (`push: ${ github.event_name != 'pull_request' }`). This validates that the Dockerfile is correct without pushing potentially unstable images.

On merge to main — the image is built, pushed with the commit SHA as a tag, scanned, and signed. The commit SHA tag means every commit produces a uniquely identified, traceable image.

22 Infrastructure Deployment Automation

Automating infrastructure deployments — applying Terraform, deploying Helm charts, updating Kubernetes manifests — closes the loop between code changes and actual running systems. Manual infrastructure changes are slow, error-prone, and leave no audit trail.

Terraform automation in CI/CD:

yaml

46 lines

```

terraform-plan:
  runs-on: ubuntu-latest
  steps:

```

```

- uses: actions/checkout@v4
- uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: 1.6.0

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${ secrets.TF_PLAN_ROLE }
    aws-region: us-east-1

- name: Terraform init
  working-directory: infrastructure/environments/staging
  run: terraform init

- name: Terraform plan
  working-directory: infrastructure/environments/staging
  run: terraform plan -out=plan.tfplan

- name: Upload plan
  uses: actions/upload-artifact@v3
  with:
    name: terraform-plan
    path: infrastructure/environments/staging/plan.tfplan

terraform-apply:
  needs: terraform-plan
  environment: staging # Requires approval
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3

    - name: Download plan
      uses: actions/download-artifact@v3
      with:
        name: terraform-plan
        path: infrastructure/environments/staging/

    - name: Terraform apply
      working-directory: infrastructure/environments/staging
      run: terraform apply plan.tfplan

```

The pattern of separating plan and apply — with human approval between — ensures that the plan output is reviewed before infrastructure changes are actually made. The plan artifact is generated in the plan job and consumed in the apply job, guaranteeing that what was reviewed is exactly what's applied.

GitOps-based deployment — for Kubernetes application deployments, GitOps (ArgoCD or Flux watching a Git repository) is often superior to pipeline-driven deployments. The pipeline updates image tags or Helm values in the GitOps repository. ArgoCD detects the change and applies it to the cluster. The pipeline doesn't need Kubernetes credentials — ArgoCD running inside the cluster handles the actual deployment.

yaml

23 lines

```
update-deployment:
  needs: build-push
  runs-on: ubuntu-latest
  steps:
    - name: Checkout GitOps repo
      uses: actions/checkout@v4
      with:
        repository: myorg/gitops-config
        token: ${ secrets.GITOPS_PAT }

    - name: Update image tag
      run: |
        sed -i 's|image: myregistry/model-server:.*|image: myregistry/model-server:sha-${ github.sha }|' \
          environments/staging/model-server/deployment.yaml

    - name: Commit and push
      run: |
        git config user.email "ci@company.com"
        git config user.name "CI Bot"
        git add -A
        git commit -m "Update model-server to sha-${ github.sha }"
        git push
```

ArgoCD detects the Git commit and syncs the cluster to match.

23 Automated Versioning & Release Pipelines

Manual versioning — a developer decides when to bump versions, manually edits version files, creates a tag, writes release notes — doesn't scale and produces inconsistent results. Automated versioning derives version numbers from commit history and creates releases without human intervention.

Conventional Commits is the foundation. When your team commits using structured commit messages — `feat:`, `fix:`, `chore:`, `docs:`, `breaking change:` — automated tooling can analyze the commit history and determine what the next semantic version should be. A commit with `feat:` since the last release indicates a minor version bump. A commit with `fix:` indicates a patch bump. A commit with `BREAKING CHANGE:` in the footer indicates a major version bump.

Release Please from Google automates the entire release workflow as a GitHub Action:

```
release-please:
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: google-github-actions/release-please-action@v4
      with:
        release-type: python
        package-name: model-server
```

When this runs after a merge to main, Release Please analyzes commits since the last release, determines the appropriate version bump, updates version files and CHANGELOG.md, and creates a "Release PR" with all these changes. When the Release PR is merged, Release Please creates the Git tag and GitHub Release automatically.

The changelog is generated from commit messages — `feat:` entries become "Features" sections, `fix:` entries become "Bug Fixes", breaking changes get prominent documentation. This creates accurate, consistently formatted release notes without manual writing.

Semantic Release is another popular tool for the same purpose, with plugins for different languages and ecosystems:

```
# .releaserc.yml
branches:
  - main
plugins:
  - "@semantic-release/commit-analyzer"
  - "@semantic-release/release-notes-generator"
  - "@semantic-release/changelog"
  - "@semantic-release/github"
  - "@semantic-release/git"
```

Version propagation — when a release is created, the version needs to propagate to all the places it's referenced. Python's `pyproject.toml`, Docker image tags, Helm chart versions, Kubernetes deployment labels. Automated release pipelines handle all of this through post-release jobs triggered by the tag creation event.

24 Blue-Green & Canary Deployment Automation

We covered Blue-Green and Canary strategies conceptually in the Deployment Strategies section. Here we focus on automating these strategies within CI/CD pipelines.

Automated Blue-Green with Kubernetes:

```

blue-green-deploy:
  runs-on: ubuntu-latest
  environment: production
  steps:
  - name: Deploy to green slot
    run: |
      # Create green deployment with new version
      kubectl set image deployment/model-server-green \
        model-server=myregistry/model-server:${{ github.sha }} \
        -n production

      # Wait for green to be ready
      kubectl rollout status deployment/model-server-green \
        -n production --timeout=10m

  - name: Run smoke tests against green
    run: |
      GREEN_IP=$(kubectl get svc model-server-green -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
      python tests/smoke_test.py --host $GREEN_IP

  - name: Switch traffic to green
    run: |
      # Update the main service selector to point to green pods
      kubectl patch service model-server \
        -p '{"spec":{"selector":{"slot":"green"}}}'

  - name: Wait and validate
    run: |
      sleep 60
      python tests/production_validation.py

  - name: Update blue slot for next deployment
    run: |
      kubectl set image deployment/model-server-blue \
        model-server=myregistry/model-server:${{ github.sha }} \
        -n production

```

Automated Canary with Argo Rollouts:

yaml

12 lines

```

canary-deploy:
  runs-on: ubuntu-latest
  steps:
  - name: Update rollout image
    run: |
      kubectl argo rollouts set image model-server \
        model-server=myregistry/model-server:${{ github.sha }}

  - name: Monitor rollout
    run: |
      kubectl argo rollouts status model-server --timeout=30m

```

Argo Rollouts handles the canary progression automatically based on its configured analysis — it promotes traffic from 5% to 20% to 50% to 100% as long as error rate and latency metrics stay within thresholds, and rolls back automatically if they don't. The pipeline just updates the image and monitors the outcome.

25 Rollback Strategies

Automated rollback is the safety net for when deployments go wrong. In high-quality CI/CD pipelines, rollback is not a manual panic response — it's an automated process that fires based on monitoring signals.

Automated deployment rollback — in your deployment job, after applying the change, watch key metrics for a defined window:

yaml

21 lines

```
deploy-and-validate:
  runs-on: ubuntu-latest
  steps:
    - name: Deploy new version
      run: kubectl set image deployment/model-server model-server=myregistry/model-server:${{ github.sha }}

    - name: Wait for rollout
      run: kubectl rollout status deployment/model-server --timeout=10m

    - name: Validate deployment health
      run: |
        # Wait 5 minutes and check error rate
        sleep 300
        ERROR_RATE=$(curl -s "http://prometheus/api/v1/query?query=rate(http_errors_total[5m])" | jq '.data.result[0].value[1]')
        if (( $(echo "$ERROR_RATE > 0.01" | bc -l) )); then
          echo "Error rate too high: $ERROR_RATE. Rolling back."
          kubectl rollout undo deployment/model-server
          exit 1
        fi
        echo "Deployment healthy. Error rate: $ERROR_RATE"
```

Kubernetes-native rollback — for Kubernetes deployments, `kubectl rollout undo` reverts to the previous ReplicaSet and is very fast. The previous image is still cached on nodes, so rollback typically completes in under a minute.

GitOps rollback — with GitOps deployments, rollback means reverting the Git commit that updated the image tag. ArgoCD detects the revert and rolls the cluster back to the previous state. This creates a clean audit trail — the rollback is visible in Git history.

Database migration rollback — the hardest rollback scenario involves database schema changes. If a deployment includes a migration that is not backward-compatible (removes a column the old code reads), rolling back the application code while the migration has run leaves the old code broken against the new schema. This is why migrations should always be backward-compatible — add columns as nullable, don't remove columns in the same release as the code change that removes references to them (use the expand-contract pattern from the Terraform section).

26 CI/CD Security Best Practices

CI/CD pipelines are one of the highest-value targets in your infrastructure. A compromised pipeline can deploy malicious code to production, exfiltrate secrets, or destroy infrastructure. Securing pipelines is as important as securing production systems.

Least privilege for pipeline credentials — the IAM role or service account used by a pipeline job should have exactly the permissions needed for that job. The linting job needs no cloud access. The build job needs registry push access. The staging deployment job needs staging cluster access. The production deployment job needs production cluster access. Separate roles for each, with no cross-environment access.

Protect secrets from pull request workflows — GitHub Actions has an important security boundary: secrets are not available to workflows triggered by pull requests from forks, because a malicious PR author could modify the workflow to exfiltrate secrets. This is correct behavior. For your own organization's PRs, configure your secrets and environments appropriately, but be aware of this boundary.

Pin action versions with SHA — using `uses: actions/checkout@v4` is convenient but the `v4` tag can be moved to point to a different commit (supply chain attack). Using the full commit SHA `uses: actions/checkout@a5ac7e51b41094c92402da3b24376905380afc29` ensures you're using exactly the code you audited.

OIDC instead of static credentials — as covered in secrets management, replacing long-lived static credentials with OIDC-based temporary credentials eliminates the most common credential leak vector in CI/CD.

Limit permissions on GitHub token — GitHub Actions automatically provides a `GITHUB_TOKEN` with broad repository permissions. Restrict it to the minimum needed:

yaml

5 lines

```
permissions:
  contents: read      # Only read repository contents
  packages: write     # Write to GitHub Packages if needed
  id-token: write     # OIDC token generation
```

Audit pipeline changes — pipeline files are code and should go through PR review. Changes to pipeline files — especially to how secrets are used, what environments they deploy to, and what approval requirements are enforced — should require review from senior engineers or security team members. Consider code owners rules that require specific approvers for `.github/workflows/` changes.

27 Observability in CI/CD

A CI/CD pipeline is infrastructure that you depend on for your entire development velocity. Without observability, you can't answer basic questions: how long do pipelines take? What's the failure rate? Where is time being spent? Which steps are getting slower over time?

Pipeline metrics — collect and track key metrics for every pipeline run. Total duration, duration per stage, success/failure rate, failure rate by stage (which stages fail most often?), time to first feedback (how long until a developer knows if their commit passed?), and frequency (how many runs per day?). These metrics, tracked over time, reveal degrading performance, flaky tests, and bottlenecks.

GitHub Actions doesn't expose metrics natively, but tools like Datadog CI Visibility, BuildPulse, and Grafana's GitHub Actions integration collect and visualize this data.

Test result tracking — upload test results in standard formats (JUnit XML) and track them in a dedicated system. Tools like BuildPulse and Datadog CI Visibility track test results across runs and identify flaky tests (tests that fail intermittently without code changes), slow tests (tests taking significantly longer than average), and newly failing tests (tests that were passing and just started failing).

Flaky tests are particularly insidious — they erode trust in CI. When developers see a red pipeline and assume it's a flaky test rather than a real failure, they start merging code without validating CI. Identifying and fixing flaky tests is a high-priority maintenance task for any team that depends on CI.

Pipeline failure alerts — when pipelines fail on main (not on a PR — PR failures are expected), that's a signal that something merged that broke the build. Alert the team immediately — a broken main pipeline means nobody can ship until it's fixed. Paging or Slack alerts for main branch pipeline failures are appropriate.

Cost observability — CI/CD compute costs money. Tracking cost per pipeline run, cost per repository, and cost trends over time helps identify expensive pipelines that might benefit from optimization. A matrix build running on 20 large runners for every PR might be appropriate for a critical library but overkill for a small service.

Trace individual pipeline runs — when a pipeline fails, diagnosing the cause should be fast. Pipeline logs should be structured (machine-parseable as well as human-readable), retain for long enough to investigate incidents (at least 90 days), and be searchable. When an on-call engineer gets paged at 2am because the production deployment pipeline failed, they should be able to find the relevant log lines within seconds, not minutes.

The thread connecting all of them is this: both Git discipline and CI/CD engineering are about creating systems that make the path from idea to production fast, safe, auditable, and reversible. Git gives you the history and collaboration foundation. CI/CD automates the quality gates and deployment mechanics. Together, done well, they create an engineering culture where shipping is easy and safe, and where mistakes are caught early and recovered from quickly.

Docker & Container Security Engineering

01 Containerization Fundamentals

Before containers existed, deploying software was genuinely painful in ways that are hard to appreciate if you haven't lived through it. You'd write an application on your laptop, running Python 3.9 with specific library versions on macOS. The staging server runs Python 3.7 on Ubuntu. Production runs Python 3.8 on CentOS. Each environment has different library versions installed, different system dependencies, different file paths, different environment variables. Your application works perfectly on your laptop and mysteriously breaks in production. The phrase "it works on my machine" became a running joke in the industry because it was so universally true and so universally frustrating.

Virtual machines were the first attempt at solving this. You package the entire operating system along with your application into a VM image. Now you have consistency — the VM is the same everywhere. But VMs are heavy. They include a full OS kernel, gigabytes of operating system files, and take minutes to start. Running 50 services means running 50 full VMs, each with its own OS consuming gigabytes of RAM just for the operating system overhead. This is expensive and slow.

Containers solve the same problem as VMs — application isolation and consistency — but with a fundamentally different approach. Instead of virtualizing the hardware and running separate OS kernels, containers share the host machine's OS kernel and use Linux kernel features to create isolated environments for each application.

The two key Linux kernel features that make containers work are namespaces and cgroups.

Namespaces provide isolation. Linux has several types of namespaces, each isolating a different aspect of the system. The PID namespace means processes inside a container can only see their own processes — the container thinks it's the only thing running on the machine, unaware of thousands of other processes on the host. The network namespace gives the container its own network stack — its own network interfaces, routing tables, and port space. A container can bind to port 8080 without conflicting with other containers also binding to port 8080, because each has its own isolated port space. The mount namespace isolates the filesystem — the container has its own view of the filesystem, completely separate from the host's filesystem. The UTS namespace isolates the hostname. The IPC namespace isolates inter-process communication. Together, these namespaces create the illusion that the container is running in its own isolated machine.

Cgroups (control groups) provide resource control. While namespaces create isolation, cgroups enforce limits. They allow the kernel to limit how much CPU, memory, network bandwidth, and disk I/O a group of processes can use. When you say a container can use at most 2 CPUs and 4GB of RAM, cgroups enforce that boundary. If the container tries to allocate more memory than allowed, the kernel's OOM killer terminates the process.

A container image is the static, immutable blueprint of a container — the filesystem snapshot that defines what's inside the container. When you run a container, the runtime creates an isolated environment from the image, adds a writable layer on top (so the running container can write files without modifying the shared image), and starts the specified process.

Because containers share the host kernel, they're dramatically lighter than VMs. Starting a container takes milliseconds instead of minutes. Running 50 containers on a machine uses a fraction of the memory of 50 VMs because there's only one OS kernel, not 50. You can run hundreds of containers on a single machine.

The consistency guarantee comes from the image itself. A container image contains everything the application needs to run: the application code, the exact runtime version, all library dependencies, configuration files. The image is built once and runs identically on any machine with a compatible container runtime — your laptop, CI/CD, staging, production. "It works on my machine" becomes "it works in this container" and the container is the same everywhere.

02 Dockerfile Best Practices

A Dockerfile is the recipe for building a container image. It's a text file containing a series of instructions that the container build tool (Docker, Buildah, Kaniko) executes in order to produce an image. Writing Dockerfiles well matters enormously — poor Dockerfiles produce large, slow-to-build, insecure images that make your entire development and deployment cycle worse.

Let's go through what good Dockerfile practice looks like and why each decision matters.

Choose your base image carefully — the `FROM` instruction is the first line of every Dockerfile and sets your starting point. The choice of base image determines the attack surface, image size, and available tooling. Using `FROM ubuntu:latest` gives you a full Ubuntu installation with hundreds of packages you almost certainly don't need. For a Python ML application, `FROM python:3.11-slim` is a much better starting point — it's Ubuntu-based but with non-essential packages removed. Better still is `FROM python:3.11-slim-bookworm` which pins a specific Debian version. Always pin specific versions, never use `latest` — `latest` changes without warning and makes your builds non-reproducible.

Pin dependency versions — both in the base image tag and in your application dependencies. Your `requirements.txt` should have exact versions (`torch==2.1.0` , not `torch ≥ 2.0`). If you don't pin versions, rebuilding your image six months from now might produce a completely different application because dependencies updated. This causes subtle bugs that are extremely hard to trace to their source.

Order instructions from least to most frequently changing — this is the single most impactful practice for build speed. Docker builds images in layers, and each instruction creates a new layer. Docker caches layers — if a layer's instruction and all its inputs haven't changed since the last build, Docker reuses the cached layer instead of running the instruction again. Critically, when any layer changes, all subsequent layers are invalidated and must be rebuilt from scratch.

What this means in practice: put instructions that change rarely (installing system dependencies, copying requirements files) at the top, and instructions that change frequently (copying your application code) at the bottom.

Bad ordering:

dockerfile

4 lines

```
FROM python:3.11-slim
COPY . /app          # Copies everything including code that changes every commit
RUN pip install -r requirements.txt # This runs every time ANY file changes
```

Good ordering:

dockerfile

5 lines

```
FROM python:3.11-slim
COPY requirements.txt /app/requirements.txt # Only this file
RUN pip install -r /app/requirements.txt     # Cached unless requirements.txt changes
COPY . /app                                 # Code changes don't invalidate pip cache
```

With the good ordering, `pip install` only re-runs when `requirements.txt` changes. When you change only application code, Docker uses the cached pip layer and only runs the final `COPY`. For a project with many dependencies, this can be the difference between a 30-second rebuild and a 15-minute rebuild.

Minimize the number of layers — each RUN instruction creates a new layer. Chaining related commands into a single RUN instruction with `&&` reduces layer count and, importantly, allows you to clean up intermediary files within the same layer, preventing them from persisting in the image:

dockerfile

7 lines

```
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        libgomp1 \
        libsndfile1 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

If you ran `apt-get update` and `apt-get install` as separate layers, and then `rm -rf /var/lib/apt/lists/*` as another, the cache files from `apt-get update` would exist permanently in the second layer even though the third layer deletes them. In a single layer, the entire operation produces a layer that never contained the cache files in the first place.

Use `.dockerignore` — just like `.gitignore` tells Git which files to exclude, `.dockerignore` tells Docker which files and directories to exclude from the build context (the set of files sent to the Docker daemon for building). Without `.dockerignore`, `COPY . /app` copies everything — your entire `.git` directory, local development configurations, test data, documentation, virtual environments. This massively increases build time and image size. A good `.dockerignore` includes: `.git`, `__pycache__`, `*.pyc`, `node_modules`, `.env`, `*.log`, `tests/`, local data directories.

Set `WORKDIR` — always set a working directory with the `WORKDIR` instruction rather than using `cd` in RUN commands. `WORKDIR` creates the directory if it doesn't exist and sets it as the working directory for all subsequent instructions. Using absolute paths avoids ambiguity and makes the Dockerfile easier to understand.

Expose the right port — `EXPOSE` doesn't actually publish a port, it's documentation that says "this container listens on this port." Always include it because it helps humans and tooling understand the container's expected interface.

Use ENTRYPOINT and CMD correctly — `ENTRYPOINT` defines the executable that always runs. `CMD` provides default arguments that can be overridden. For a model server, `ENTRYPOINT ["python", "-m", "model_server"]` and `CMD ["--port", "8080"]` means you can override the port at runtime without overriding the entire command. Use JSON array form (`["python", "app.py"]`) rather than string form (`python app.py`) — the JSON form uses `exec` directly, while the string form uses a shell, which creates an extra shell process and can cause issues with signal handling.

03 Multi-Stage Builds

Multi-stage builds solve one of the most common problems in container image construction: the tension between needing a rich build environment and wanting a minimal runtime image.

To understand the problem, think about building a compiled application. You need compilers, build tools, development headers, and test frameworks to build and test the application. But to run it in production, you need only the compiled binary and its runtime dependencies. If you use a single Dockerfile that includes everything needed to build the application, your final image contains all those build tools that you only needed during compilation — making your production image gigabytes larger than necessary.

For ML systems, this problem appears in a different form. To build a Python ML package that has native extensions (like `numpy`, `scipy`, or `PyTorch`), you might need C compilers, CUDA development headers, and build toolchains. To train or serve the model, you need only the installed Python packages, not the compilers used to build them.

Multi-stage builds let you have multiple `FROM` statements in a single Dockerfile, each starting a new build stage. You can copy artifacts from one stage to another, and only the final stage ends up in the produced image.

Here's a concrete example for an ML inference service:

dockerfile

34 lines

```
# Stage 1: Builder — has all tools needed to install dependencies
FROM python:3.11 AS builder

WORKDIR /build

# Install build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    gcc \
    g++ \
    && rm -rf /var/lib/apt/lists/*

# Copy and install Python dependencies into a specific location
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/build/packages -r requirements.txt

# Stage 2: Runtime — minimal image with only what's needed to run
```

```
FROM python:3.11-slim AS runtime

WORKDIR /app

# Copy only the installed packages from the builder stage
COPY --from=builder /build/packages /usr/local/lib/python3.11/site-packages

# Copy application code
COPY src/ /app/src/

# Run as non-root user
RUN useradd --create-home --shell /bin/bash appuser
USER appuser

ENTRYPOINT ["python", "-m", "src.model_server"]
```

The builder stage uses the full `python:3.11` image and installs `build-essential` with compilers. It installs all Python dependencies, some of which need compilation. The runtime stage starts fresh from `python:3.11-slim` — a much smaller image — and copies only the installed packages from the builder. The compilers, build tools, and intermediate build artifacts are not in the final image at all.

Test stages — multi-stage builds also work well for running tests within the image build:

dockerfile

10 lines

```
FROM python:3.11-slim AS base
# ... install dependencies ...

FROM base AS test
COPY tests/ /app/tests/
RUN python -m pytest /app/tests/

FROM base AS production
# ... copy only production code, no test files ...
```

You can build just the test stage during CI (`docker build --target test`) to validate the application, and build the production stage for deployment. Tests run in the same environment as production, eliminating "tests pass but production fails" scenarios.

Build arguments across stages — you can pass build arguments to control behavior across stages:

dockerfile

4 lines

```
ARG MODEL_VERSION=latest
FROM base AS production
RUN python download_model.py --version=$MODEL_VERSION
```

Caching with multi-stage builds — BuildKit (Docker's modern build engine, now default) is smarter about caching with multi-stage builds. Stages that haven't changed don't get rebuilt even if other stages change. If your dependencies didn't change, the builder stage is cached and reused, even if your application code changed in the runtime stage.

The result of well-implemented multi-stage builds for ML services is often dramatic — images that would be 8GB as single-stage become 2GB or less, because all the CUDA development headers, compilers, and build artifacts are left in the builder stage that never makes it to the final image.

04 Image Layer Optimization

Every instruction in a Dockerfile that produces filesystem changes creates a new layer. Understanding layers — what they are, how they're stored, and how they affect image size and build performance — lets you make deliberate decisions that result in smaller, faster images.

A layer is a diff — the set of filesystem additions, modifications, and deletions that an instruction produces relative to the previous layer. Docker stores images as a stack of these diffs. When you run a container, Docker overlays all the layers into a unified filesystem view using a union filesystem (overlayfs in modern systems). The container sees a single unified filesystem, but the storage driver manages it as a stack of read-only layers with a writable layer on top.

Several important properties of layers:

Layers are immutable — once created, a layer never changes. If you modify a file in a later layer, the original file still exists in the earlier layer and is shadowed by the modification. If you delete a file in a later layer, the file still occupies space in the earlier layer — it's just hidden. This is why cleanup must happen in the same RUN instruction as the installation that created the files. If you install something in one layer and delete the cache in the next layer, the cache still occupies space in the earlier layer.

Layers are shared across images — if you have 10 images that all start with `FROM python:3.11-slim`, they all share the same base image layers. Only one copy of those layers is stored on disk and in registry. This makes pulling related images fast — layers that are already cached locally don't need to be downloaded. For ML teams where everyone is using similar base images, this sharing can save significant storage and bandwidth.

Layer size matters for pull times — when Kubernetes pulls an image to a new node (because a Pod is scheduled there for the first time), it pulls each layer. Large images on nodes that don't have them cached mean slow Pod startup. For ML serving where fast scaling is important, minimizing image size directly reduces the time from "need more Pods" to "Pods are serving traffic."

Practical layer optimization techniques:

Combine all related operations — as mentioned in Dockerfile best practices, chain related commands with `&&` so they're a single layer:


```
# Bad: two layers, cache exists in layer 1 even after layer 2 deletes it
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# Good: one layer, only the installed package remains
RUN apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*
```

Remove what you don't need in the same layer you create it — Python `.pyc` files and `__pycache__` directories, pip's download cache, npm's cache, apt's package lists — all of these are created during installation and should be removed in the same RUN instruction.

Install only what you actually need — every package you install grows the image and adds potential vulnerabilities. For system packages, use `--no-install-recommends` with apt to install only the requested package without its "recommended" packages (which are often optional and large). Audit your dependencies regularly and remove ones no longer used.

Analyze your image — tools like `dive` are invaluable for understanding what's in each layer:

```
dive myregistry/model-server:v2.1.0
```

Dive shows you each layer, what's in it, what's changed relative to the previous layer, and how much space each layer uses. This helps you identify unexpectedly large layers, files that shouldn't be in the image, and opportunities for optimization. Running dive regularly during image development is the fastest way to understand where size is coming from and where to optimize.

Layer caching in CI/CD — for CI/CD builds, explicitly cache the Docker layer cache between builds. Without this, every CI build starts from scratch and re-downloads all dependencies. With BuildKit cache mounting and registry cache layers, CI builds can achieve near-local build speeds.

05 Distroless & Minimal Base Images

The base image you choose is the foundation of your container's security posture. A base image is not neutral — every package included in it is a potential vulnerability. A package you never use is a vulnerability that can never benefit you. Distroless and minimal base images reduce your attack surface by including only what's actually needed to run your application.

Let's understand the spectrum from largest to smallest.

Full distribution images like `ubuntu:22.04` or `debian:bookworm` include a complete Linux userspace — shell (`/bin/bash`), package manager (`apt`), common utilities (`curl`, `wget`, `tar`, `gzip`, `grep`, `find`), system libraries, and more. These are large (70-120MB compressed), have hundreds of installed packages, and present a large attack surface. Most of this is useful for interactive debugging but serves no purpose in a production container. An attacker who compromises your application and gets code execution inside the container has a full shell and utilities to work with.

Slim variants like `python:3.11-slim` or `debian:bookworm-slim` strip out many packages — documentation, development headers, unnecessary utilities — but retain the shell and package manager. They're significantly smaller (30-50MB compressed) and have fewer vulnerabilities, while still being relatively easy to work with during development and debugging.

Alpine images like `python:3.11-alpine` are based on Alpine Linux, an extremely minimal distribution using musl libc instead of glibc. Alpine images are tiny (5-10MB) and have a very small package count. The tradeoff is compatibility — some Python packages that have native extensions assume glibc and don't work correctly (or don't have musl-compatible wheels), requiring compilation from source in your build stage. For ML libraries like numpy, scipy, and PyTorch, Alpine can be problematic and the compilation time penalty is significant.

Distroless images take minimalism further than Alpine. Distroless images, maintained by Google, contain only the application runtime and its dependencies — no shell, no package manager, no utilities. There is nothing to execute except your application's runtime. A distroless Python image contains the Python interpreter and standard library, but not bash, not apt, not curl — nothing else.

dockerfile

11 lines

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --target=/app/packages -r requirements.txt
COPY src/ /app/src/

FROM gcr.io/distroless/python3-debian12 AS production
COPY --from=builder /app /app
WORKDIR /app
ENTRYPOINT ["python3", "src/model_server.py"]
```

The security benefits are significant. If an attacker exploits a vulnerability in your model server and gains code execution, they have a Python interpreter and nothing else. No shell to run commands, no curl to download tools, no utilities to explore the filesystem. The lateral movement options from a compromised distroless container are dramatically limited compared to a full Ubuntu container.

The operational tradeoff is debugging difficulty. When something goes wrong in a distroless container, you can't `kubectl exec` into it and run shell commands to investigate. You need to rely on logs, metrics, and traces for debugging, which is better practice anyway for production systems. For situations where you need to debug a specific container, you can use Kubernetes's ephemeral containers feature to attach a temporary debugging container with full tooling to a running distroless container without modifying the container itself.

Chainguard images are a newer option from the Chainguard company, providing extremely minimal, continuously rebuilt images with a strong security focus. They typically have zero known CVEs at build time because they're rebuilt frequently with the latest patched packages and include only what's needed.

For ML specifically — ML images are notoriously large because of PyTorch, TensorFlow, and CUDA. Going fully distroless with ML libraries is challenging because of complex native dependencies. A practical approach is multi-stage builds with slim bases for the final image, combined with the layer optimization

techniques from the previous section. Reducing from 8GB to 3GB still represents a massive improvement in pull time, attack surface, and storage cost, even if you can't reach the 50MB distroless ideal.

06 Non-Root Containers

By default, processes inside a container run as root (UID 0) unless explicitly configured otherwise. This is one of the most common security misconfigurations in containerized applications, and it's important to understand why it matters and how to fix it.

First, let's understand container root vs host root. The root user inside a container is not the same as the root user on the host — namespaces provide isolation. But this isolation is imperfect. Container escape vulnerabilities have been found and will continue to be found in container runtimes and the Linux kernel. If an attacker compromises your application and gets code execution inside the container running as root, and they then find a container escape vulnerability, they arrive on the host machine as root. Host root is unlimited power — they can read any file, kill any process, install anything, access any other container.

If your application runs as a non-root user inside the container and the attacker exploits the same application vulnerability, they have code execution as a low-privilege user. A container escape still potentially exists, but they arrive on the host as a low-privilege user with dramatically fewer capabilities. Defense in depth means making each level of exploitation harder, and running as non-root is a low-cost, high-value layer.

Beyond security, there are operational reasons to run as non-root. Many organizations' security policies require it. Kubernetes security policies (PodSecurityAdmission with Restricted profile) can prevent root containers from being scheduled. Compliance frameworks may mandate it.

Creating a non-root user in a Dockerfile:

dockerfile

23 lines

```
FROM python:3.11-slim

WORKDIR /app

# Install dependencies as root (before switching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY src/ /app/src/

# Create a non-root user and group
RUN groupadd --gid 1001 appgroup && \
    useradd --uid 1001 --gid appgroup --shell /bin/bash --create-home appuser

# Change ownership of the app directory
RUN chown -R appuser:appgroup /app

# Switch to non-root user
USER appuser
```

```
ENTRYPOINT ["python", "-m", "src.model_server"]
```

Alternatively, using the numeric form of the USER instruction which doesn't require the user to exist in the image's `/etc/passwd`:

dockerfile

2 lines

```
USER 1001:1001
```

Using numeric UIDs (especially ones above 10000 to avoid potential conflicts with system users) is actually preferred in Kubernetes environments where you can enforce non-root by checking UIDs rather than usernames.

Read-only filesystem complements non-root nicely. If the container filesystem is mounted read-only, even a root process inside the container can't modify system files. In Kubernetes:

yaml

6 lines

```
securityContext:
  readOnlyRootFilesystem: true
  runAsNonRoot: true
  runAsUser: 1001
  runAsGroup: 1001
```

For applications that need to write files (temporary files, logs, model downloads), you explicitly mount writable volumes at specific paths rather than making the entire filesystem writable. This forces deliberate decisions about where writes happen and limits what a compromised process can modify.

Dropped capabilities — Linux capabilities break root's monolithic privilege into discrete units. Even when running as root inside a container, you can drop all capabilities except the ones your application genuinely needs (which is usually none). Adding this to your Kubernetes pod spec:

yaml

5 lines

```
securityContext:
  capabilities:
    drop:
      - ALL
```

Drops all Linux capabilities from the container process. A container with no capabilities has far less power even if it's running as root.

File permissions matter — when running as non-root, ensure your application files have appropriate permissions. Configuration files should be readable but not necessarily writable. Log directories the application writes to should be owned by the application user. Model weight files loaded at startup should be readable. Getting permissions right during the Dockerfile build is important — changing permissions at container startup is slow and error-prone.

07 Health Checks in Containers

Kubernetes has its own probe system (covered in the Kubernetes section), but Docker also has a native health check mechanism that operates at the container level, independent of any orchestrator. Understanding Docker health checks matters because they work in environments beyond Kubernetes — Docker Compose, Docker Swarm, and standalone Docker — and they provide a foundation that Kubernetes can build on.

The `HEALTHCHECK` instruction in a Dockerfile defines a command that Docker runs periodically inside the container to determine if it's healthy. Docker tracks health status and uses it for various purposes: `docker ps` shows health status, Docker Swarm uses it for service management, and some deployment tools use it to gate traffic.

dockerfile

6 lines

```
HEALTHCHECK --interval=30s \  
    --timeout=10s \  
    --start-period=60s \  
    --retries=3 \  
CMD curl --fail --silent http://localhost:8080/health || exit 1
```

The parameters: `--interval=30s` — how often to run the health check. Too frequent and you're adding unnecessary load. Too infrequent and you take longer to detect failures. `--timeout=10s` — how long to wait for the health check command to complete before considering it failed. Should be shorter than the interval. `--start-period=60s` — grace period after container start during which health check failures don't count against the retry limit. This is for applications (like model servers loading large weights) that take time to initialize. Setting this too short causes containers to be marked unhealthy during normal startup. `--retries=3` — how many consecutive failures before the container is marked unhealthy.

The health check command must exit 0 for healthy, exit 1 for unhealthy. The example uses `curl --fail` which exits non-zero on HTTP error responses. For services without curl (distroless images), you use an alternative like a dedicated health check binary compiled into the image, or a Python one-liner:

dockerfile

3 lines

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \  
CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')" || exit 1
```

What your health endpoint should check — a health endpoint that just returns 200 OK unconditionally is nearly useless. A good health endpoint verifies that the application is actually functional. For a model server, this means: can I access the model in memory? Is the model loaded and can it make a simple inference? Is the database connection pool healthy? Are critical dependencies reachable? A health check that validates actual functionality catches deadlocks, resource exhaustion, and partial failures that a simple process-alive check misses.

Liveness vs readiness in Docker — Docker's `HEALTHCHECK` is a single signal. Kubernetes's split into liveness and readiness (and startup) is more nuanced and more useful. In Docker-only environments, the `HEALTHCHECK` does double duty — it both signals whether the container is alive and whether it can receive

traffic. In Kubernetes, the HEALTHCHECK is used less because Kubernetes's probe system is richer, but it still provides useful metadata visible in container state.

Health check in Docker Compose — Docker Compose uses health checks for service dependency management:

yaml

15 lines

```
services:
  model-server:
    build: .
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 60s

  inference-gateway:
    depends_on:
      model-server:
        condition: service_healthy # Don't start until model-server is healthy
```

This ensures the inference gateway only starts after the model server has passed its health check — critical for ML services where dependent services fail if they start before the model is loaded.

08 Container Networking Basics

Container networking is one of the most conceptually tricky aspects of containers because there are multiple networking contexts happening simultaneously — the host machine's network, the container's private network, bridge networks, and overlay networks — and understanding which applies in which situation requires some foundation building.

Let's build up from scratch.

Default container networking — when Docker starts on a machine, it creates a virtual network bridge called `docker0`. This bridge has an IP address (typically `172.17.0.1`) and acts as a gateway for containers. By default, every container gets a virtual network interface connected to this bridge and an IP address in the bridge's subnet (like `172.17.0.2`, `172.17.0.3`, etc.). Containers on the same bridge can communicate with each other using these IPs. The host machine can reach containers via these IPs. But these IPs are private to the host — machines outside the host cannot reach them directly.

This default bridge network is fine for basic experimentation but has a problem for multi-container applications: containers on the default bridge can only reference each other by IP, not by name. Since container IPs are assigned dynamically and change when containers restart, relying on IPs is fragile.

User-defined bridge networks solve the naming problem. When you create a custom network and put containers on it, Docker provides automatic DNS resolution — containers can reach each other by container name or service name. If your model server container is named `model-server`, your gateway container can

connect to it at `http://model-server:8080` and Docker's built-in DNS resolves `model-server` to the correct container IP automatically.

bash

4 lines

```
docker network create ml-network
docker run --network=ml-network --name model-server myimage/model-server
docker run --network=ml-network --name gateway myimage/gateway
```

Now the gateway container can connect to `model-server:8080` regardless of what IP the model server container happens to have.

Port publishing — containers have their own network namespace with their own port space. A container process binding to port 8080 is binding to port 8080 in the container's namespace, not the host's port 8080. To make the container accessible from outside the host (or from the host itself without knowing the container IP), you publish ports with the `-p` flag: `-p 9090:8080` publishes the host port 9090 and forwards traffic to container port 8080. Host port 9090 is now accessible from anywhere the host is reachable, and traffic arrives at the container on port 8080.

None network — `--network=none` gives the container no network access whatsoever — no interfaces except the loopback. This is for containers that process data from volumes and don't need any network access. Maximum network isolation.

Host network — `--network=host` removes network namespace isolation — the container shares the host's network namespace directly. There's no bridge, no NAT, no port publishing — the container's processes bind directly to the host's network interfaces. This maximizes networking performance (no virtual network overhead) but eliminates isolation — the container can access any port on the host and vice versa without any mapping. Only use this when you have specific performance requirements and understand the security implications.

Container-to-container communication patterns — when two containers need to communicate, the recommended approach is user-defined bridge networks with DNS. Avoid linking (an older Docker feature that's deprecated), avoid using container IPs directly (they change), and avoid `--network=host` for isolation reasons.

DNS in containers — by default, containers use the host's DNS configuration (inherited from `/etc/resolv.conf`) for external name resolution. Docker also intercepts DNS queries for container names on user-defined networks and resolves them internally. You can configure custom DNS servers for containers if your application needs to resolve internal company DNS names.

09 Docker Compose for Multi-Service Apps

Most real ML applications are not a single container — they're a collection of services that need to work together. A model server, a feature store, a Redis cache, a PostgreSQL database, a monitoring agent, an API gateway. During development and testing, spinning up all of these manually with individual `docker run` commands is tedious and error-prone. Docker Compose solves this by defining an entire multi-service application in a single YAML file and managing it with simple commands.

A Docker Compose file describes: what services (containers) make up the application, how to build or pull each service's image, what environment variables each service needs, what ports to publish, what volumes to mount, what networks to create, and what dependencies exist between services.

A realistic Docker Compose file for a local ML development environment:

yaml

92 lines

```
version: "3.9"

services:
  model-server:
    build:
      context: .
      dockerfile: Dockerfile
      target: development      # Build the development stage
    ports:
      - "8080:8080"
    volumes:
      - ./src:/app/src        # Mount source code for live reload
      - model-weights:/app/models
    environment:
      - LOG_LEVEL=DEBUG
      - FEATURE_STORE_URL=redis://feature-store:6379
      - MODEL_PATH=/app/models/current
    depends_on:
      feature-store:
        condition: service_healthy
      mlflow:
        condition: service_healthy
    networks:
      - ml-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 60s

  feature-store:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - feature-data:/data
    networks:
      - ml-network
    healthcheck:
```



```
test: ["CMD", "redis-cli", "ping"]
interval: 10s
timeout: 5s
retries: 3
```

mlflow:

```
image: ghcr.io/mlflow/mlflow:v2.8.0
ports:
  - "5000:5000"
environment:
  - MLFLOW_BACKEND_STORE_URI=postgresql://mlflow:mlflowpass@db/mlflow
  - MLFLOW_ARTIFACT_ROOT=/mlflow/artifacts
volumes:
  - mlflow-artifacts:/mlflow/artifacts
depends_on:
  db:
    condition: service_healthy
networks:
  - ml-network
command: mlflow server --host 0.0.0.0 --port 5000
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

db:

```
image: postgres:15-alpine
environment:
  - POSTGRES_USER=mlflow
  - POSTGRES_PASSWORD=mlflowpass
  - POSTGRES_DB=mlflow
volumes:
  - postgres-data:/var/lib/postgresql/data
networks:
  - ml-network
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U mlflow"]
  interval: 10s
  timeout: 5s
  retries: 5
```

volumes:

```
model-weights:
feature-data:
mlflow-artifacts:
postgres-data:
```

```
networks:
  ml-network:
    driver: bridge
```

With this file, `docker compose up` starts all four services in the correct order (dependencies first), `docker compose logs -f model-server` streams model server logs, `docker compose down` stops everything cleanly, and `docker compose down -v` stops everything and removes volumes (for a clean slate).

Environment-specific overrides — Docker Compose supports override files that layer on top of the base configuration. `docker-compose.override.yml` is automatically merged with `docker-compose.yml`. You can have `docker-compose.production.yml` with production-specific settings and apply it with `docker compose -f docker-compose.yml -f docker-compose.production.yml up`. This pattern avoids duplicating the entire configuration for each environment.

Using Compose for CI testing — one of the most valuable uses of Docker Compose is running integration tests in CI that require real services:

yml

13 lines

```
# docker-compose.test.yml
services:
  test-runner:
    build:
      context: .
      target: test
    depends_on:
      model-server:
        condition: service_healthy
    command: pytest tests/integration/
    networks:
      - ml-network
```

`docker compose -f docker-compose.yml -f docker-compose.test.yml run test-runner` starts all services and runs integration tests against real running services, then exits with the test exit code. No mocking, no stubs — real integration tests against real services.

Compose vs Kubernetes — Docker Compose is for development and testing environments, not production. It runs on a single machine, has no built-in high availability, no automatic scaling, and no advanced health-based scheduling. Production workloads go to Kubernetes. The practical workflow: develop with Docker Compose locally, test with Docker Compose in CI, deploy with Kubernetes in production.

10 Container Image Scanning

Every container image you build is a collection of software — your application code, language runtimes, system libraries, and base OS utilities. Any of these components can have known security vulnerabilities (CVEs — Common Vulnerabilities and Exposures). Image scanning is the automated process of checking every component in your image against databases of known vulnerabilities and reporting what it finds.

The CVE ecosystem: when a security researcher discovers a vulnerability in a software package, they report it responsibly, it gets assigned a CVE identifier (like CVE-2021-44228, the Log4Shell vulnerability), and it's published in the National Vulnerability Database with a severity score (Critical, High, Medium, Low) and affected version ranges. Image scanners check every package in your image against this database.

How scanners work: they extract the software bill of materials from your image (the complete list of every installed package and version across every layer), look up each package-version pair against CVE databases, and report which packages have known vulnerabilities and how severe they are. This happens in seconds and catches vulnerabilities that would otherwise require manual tracking of every security bulletin for every package in your image.

Trivy is the most widely used open-source scanner. Running it is straightforward:

bash

2 lines

```
trivy image myregistry/model-server:v2.1.0
```

It produces output showing every vulnerability found, the affected package, the severity, the installed version, and the fixed version (if available). You can output this in various formats (table, JSON, SARIF) for integration with different tools.

Integrating scanning into CI/CD — scanning should happen automatically in your pipeline, not as an afterthought. After building an image and before pushing to the registry or deploying, run the scanner and gate the pipeline on the results:

yaml

9 lines

```
# GitHub Actions example
- name: Scan image for vulnerabilities
  run: |
    trivy image \
      --severity CRITICAL,HIGH \
      --exit-code 1 \
      --no-progress \
      myregistry/model-server:${{ github.sha }}
```

`--exit-code 1` causes Trivy to exit with a non-zero code if vulnerabilities above the specified severity are found, failing the pipeline. `--severity CRITICAL,HIGH` means only Critical and High severities block the pipeline (Medium and Low are reported but don't fail the build).

Vulnerability management beyond just scanning — scanning gives you a list, but you still need a process for what to do with it. Critical vulnerabilities with known exploits need immediate attention — update the affected package or base image. High vulnerabilities should be addressed on a defined timeline (typically within a week). Medium and Low can go through normal maintenance cycles.

The primary remediation is updating. Most vulnerabilities in base images are fixed in newer base image versions — simply updating your `FROM python:3.11-slim` to a newer patch version often eliminates dozens of vulnerabilities. For vulnerabilities in your own dependencies, update the dependency to a fixed version.

For vulnerabilities in packages you don't actually use (they came in as transitive dependencies), removing the unnecessary package is the cleanest fix.

False positives and accepted risks — some reported vulnerabilities don't actually affect your application. A vulnerability in a C library that's only exploitable through a code path your application never calls is technically a CVE but not actually a risk in practice. Most scanners support `.trivyignore` or equivalent files where you document acknowledged vulnerabilities with reasons:

diagram

3 lines

```
# CVE-2023-XXXXX - Affects LDAP functionality in libssl, not used in our application
CVE-2023-XXXXX
```

This prevents the same "known false positive" from blocking your pipeline every build while maintaining a record of why it was accepted.

Continuous scanning of registry images — scanning at build time is necessary but not sufficient. New CVEs are published constantly. An image you scanned six months ago as clean may have dozens of critical vulnerabilities today because new vulnerabilities were discovered in packages that were already installed. Your container registry (ECR, GCR, Docker Hub with Scout, Harbor) should continuously re-scan stored images and alert when new vulnerabilities are discovered in production images.

11 Software Bill of Materials (SBOM)

We covered SBOMs conceptually in the Security section, but here we'll focus on the container-specific mechanics — how SBOMs are generated for container images, what they contain, how they're stored and used, and why they matter for ML infrastructure specifically.

A container image SBOM is a complete, machine-readable inventory of every software component inside the image — every OS package installed, every Python library, every system library, every file that came from a package. This inventory is structured data that can be searched, queried, and analyzed programmatically.

Generating SBOMs — tools like Syft and Trivy can generate SBOMs from container images:

bash

6 lines

```
# Generate SBOM with Syft in SPDX format
syft myregistry/model-server:v2.1.0 -o spdx-json > model-server-sbom.spdx.json

# Generate SBOM with Trivy in CycloneDX format
trivy image --format cyclonedx myregistry/model-server:v2.1.0 > model-server-sbom.cdx.json
```

The output is a structured document listing every component. For a Python ML image, this includes: the base OS packages (hundreds of packages from the Debian base image), Python itself, every Python package from your requirements.txt, and all their transitive dependencies.

SBOM formats — two formats have emerged as standards. SPDX (Software Package Data Exchange) is an ISO standard originally developed by the Linux Foundation. CycloneDX is a newer format from OWASP with a strong focus on security use cases. Both are JSON-based and widely supported by security tools.

Generating both from your pipeline gives maximum compatibility with different consumers.

Attaching SBOMs to images — rather than storing SBOMs as separate files that can get separated from their images, attach the SBOM directly to the image in the registry using OCI artifacts:

bash

4 lines

```
# Attach SBOM to image in registry using cosign
cosign attach sbom --sbom model-server-sbom.spdx.json \
  myregistry/model-server:v2.1.0
```

Now the SBOM is discoverable alongside the image and retrieved with it. Tools that process your image can automatically find and use its SBOM without separate coordination.

Querying SBOMs at vulnerability disclosure time — the primary value of SBOMs is the ability to answer "are we affected?" quickly when a new critical vulnerability is announced. Instead of manually checking each service, you query your SBOM database:

bash

3 lines

```
# Find all images containing a specific vulnerable package
grype db diff && grype sbom:model-server-sbom.spdx.json
```

This immediately tells you whether your model server image contains the vulnerable component and which version. With SBOMs for all your images in a central store, you can query across your entire fleet in seconds.

License compliance with SBOMs — SBOMs include license information for every component. This is valuable for compliance — ensuring you don't have GPL-licensed components in a proprietary product, identifying LGPL dependencies that have specific requirements, or auditing for commercially incompatible licenses. Automated license policy enforcement against SBOMs catches these issues before they become legal problems.

For ML models — SBOMs for ML systems should extend beyond the container to include the model itself. What training data was used? What framework version trained it? What preprocessing was applied? This is model provenance documentation — increasingly required by AI governance regulations — and it's the ML analog of an SBOM for software. Connecting the model's provenance document to the container's SBOM gives you end-to-end traceability from raw data to deployed prediction.

12 Image Signing & Verification

You build a container image in your CI/CD pipeline, push it to your registry, and your deployment infrastructure pulls and runs it. How does your deployment infrastructure know that the image it's pulling is the same image your pipeline built? How does it know the image wasn't tampered with in transit, that the registry wasn't compromised and the image replaced with a malicious one, or that someone didn't manually push a backdoored image to your registry?

Without image signing, it doesn't. Your deployment infrastructure accepts any image from the configured registry, trusting that the registry is authoritative. Image signing adds cryptographic verification — the pipeline signs the image after building it, and the deployment infrastructure verifies the signature before running it. Unsigned images are rejected.

How signing works — cryptographic signing uses asymmetric key pairs. The signer has a private key kept secret. The verifier has the corresponding public key (can be widely shared). The signer creates a cryptographic signature of the image's content hash using the private key. Anyone with the public key can verify the signature — confirming that someone with the private key signed this exact image. If even one byte of the image changed after signing, verification fails.

Cosign from the Sigstore project is the modern standard for container image signing. It integrates with OCI registries and supports multiple key management strategies.

Signing with Cosign using a key pair:

bash

9 lines

```
# Generate a key pair (private key stored securely, public key distributed)
cosign generate-key-pair

# Sign the image after pushing to registry
cosign sign --key cosign.key myregistry/model-server:v2.1.0

# Verify the signature
cosign verify --key cosign.pub myregistry/model-server:v2.1.0
```

The signature is stored in the same registry as the image (as an OCI artifact attached to the image manifest), so no separate storage is needed.

Keyless signing with Sigstore — managing signing keys is itself a security challenge. If the private key is compromised, an attacker can sign malicious images. Keyless signing with Sigstore's Fulcio certificate authority eliminates long-lived signing keys entirely. Instead, the CI/CD pipeline authenticates with its OIDC identity (GitHub Actions has an OIDC token, GCP service accounts have tokens, etc.), and Sigstore issues a short-lived certificate tied to that identity. The signature is bound to the identity (<https://github.com/myorg/myrepo/.github/workflows/build.yml@refs/heads/main>) rather than a key. Verification checks that the image was signed by the expected CI/CD identity, not just any holder of a key.

bash

9 lines

```
# Keyless signing in GitHub Actions
cosign sign --yes myregistry/model-server:${{ github.sha }}

# Verification checking identity
cosign verify \
  --certificate-identity-regexp "^https://github.com/myorg/myrepo/" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
  myregistry/model-server:v2.1.0
```

Enforcing signature verification in Kubernetes — signing is only useful if deployment infrastructure enforces verification. In Kubernetes, this is done with admission controllers. Connaisseur and Policy Controller (from Sigstore) are admission webhooks that intercept every Pod creation request and verify that the container

images pass your signature policy before allowing the Pod to be scheduled. If an image is unsigned or the signature doesn't match your policy, the Pod creation is rejected.

Transparency log — Sigstore's Rekor is a public, immutable transparency log where all signatures are recorded. Every signature your pipeline creates is logged in Rekor with a timestamp. This creates an auditable record of every image signing event — you can prove when an image was signed, by what identity, and verify that the signature hasn't been backdated or fabricated.

13 Secure Registry Practices

Your container registry is where all your images live — the authoritative source from which all deployments pull. It's a high-value target: compromising the registry and replacing images with malicious versions would let an attacker run arbitrary code in every environment that pulls those images. Securing the registry is critical.

Use a private registry — public Docker Hub is appropriate for public open-source images, not for proprietary ML models, internal applications, or images that contain any embedded configuration. Run a private registry — AWS ECR, Google Artifact Registry, Azure Container Registry, GitHub Container Registry, or self-hosted Harbor. Private registries require authentication to pull images, preventing unauthorized access.

Registry authentication — every client (developer machines, CI/CD systems, Kubernetes clusters) must authenticate to pull from the private registry. In cloud-native environments, this is typically done through cloud IAM: ECR uses AWS IAM roles, Artifact Registry uses GCP service accounts. Kubernetes nodes in EKS get an IAM role that allows ECR pulls without storing credentials. This is far better than storing long-lived registry credentials as secrets.

Immutable tags — image tags are mutable by default. If you tag an image `v2.1.0` and push it, then push a different image with the same tag `v2.1.0`, the tag now points to the new image. This means the same tag can refer to different images over time, making deployments non-deterministic and making it impossible to guarantee that what's in production matches what was tested. Enable immutable tags in your registry — once a tag is pushed, it cannot be overwritten. If you need to update, create a new version tag. Immutable tags ensure that deploying `v2.1.0` today and deploying `v2.1.0` six months from now get the same image.

Reference images by digest — even with immutable tags, best practice is to reference images by their SHA256 digest in production deployments. A digest is content-addressed — it's derived from the image content, so if the content changes, the digest changes. Referencing by digest guarantees you get exactly the image you built and tested, not just the current image behind that tag.

yaml

6 lines

```
# Tag reference (the tag could theoretically be moved)
image: myregistry/model-server:v2.1.0

# Digest reference (immutably refers to a specific image)
image: myregistry/model-server@sha256:7d9c9b6f1a2e3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c
```

Access control — apply least privilege to registry access. CI/CD systems that build and push images need push access. Kubernetes clusters that pull images need pull-only access. Individual developers might need pull access for debugging but shouldn't need push access. Use separate credentials or IAM policies for push vs pull operations.

Image retention policies — without retention policies, registries accumulate images indefinitely. An active development team can generate thousands of images per month. Implement automated retention: keep the last N versions of each image, keep all images tagged with semantic versions, delete untagged images after N days. This manages storage costs and reduces the surface area of images that need vulnerability scanning.

Vulnerability scanning at the registry level — as mentioned in the scanning section, many registries support continuous scanning of stored images. Enable this and configure alerts when new critical vulnerabilities are found in images that are currently deployed. ECR Basic Scanning and Enhanced Scanning, Artifact Registry's vulnerability scanning, and Harbor's built-in Trivy integration all provide this capability.

Audit logging — enable audit logging on your registry. Every push, pull, tag creation, and deletion should be logged with the authenticating identity, timestamp, and what was accessed. This creates an audit trail for security investigations and compliance. If an unauthorized image appeared in your registry, audit logs tell you who pushed it and when.

Network-level restrictions — in addition to authentication, restrict which networks can access the registry. Your registry should not be accessible from arbitrary internet sources — restrict access to your corporate network, your cloud VPC, and specific CI/CD runner IPs. This means even if credentials are stolen, they can only be used from authorized network locations.

14 Runtime Container Security

Everything discussed so far — secure Dockerfiles, minimal base images, image scanning, signing — addresses security of the image before it runs. Runtime container security is about monitoring and protecting containers while they're actually executing in production. This is where the rubber meets the road: your container is running in production, processing real requests, and potentially being actively targeted by attackers.

Understanding what runtime security protects against — by the time a container is running in production, it's passed all your pre-deployment checks. Runtime threats come from different vectors: a vulnerability in your application code is exploited by malicious user input, a previously unknown vulnerability (zero-day) is exploited in a library your scanner didn't know was vulnerable, a developer's account is compromised and someone is trying to use running containers for lateral movement, or a dependency you pulled in had malicious code that activates at runtime.

Seccomp profiles — syscall filtering is one of the most powerful runtime security controls available. Linux applications communicate with the kernel through system calls (syscalls). A typical container process might legitimately use 50-100 different syscalls. But Linux has over 400 syscalls, many of which your application will never call. Seccomp (Secure Computing Mode) allows you to define a whitelist of syscalls a container is allowed to make. Any attempt to call a syscall not in the whitelist results in the process being killed.

Docker has a default seccomp profile that blocks the most dangerous syscalls. Kubernetes allows you to apply custom seccomp profiles:

yaml

4 lines

```
securityContext:
  seccompProfile:
    type: RuntimeDefault # Use the container runtime's default seccomp profile
```

For maximum security, create a custom profile allowing only the specific syscalls your application needs. Tools like `strace` and `inspektor-gadget` help you determine which syscalls your application actually uses.

AppArmor and SELinux — these are Mandatory Access Control (MAC) systems that enforce security policies at the OS level, outside the container. AppArmor profiles define what files a process can access, what capabilities it can use, and what network operations it can perform. If malware inside your container tries to read `/etc/shadow` (the password database) or execute unexpected binaries, AppArmor blocks it regardless of the process's permissions. These protections exist at the kernel level, below the container runtime — a container escape exploit that circumvents container namespace isolation still hits AppArmor/SELinux policy enforcement.

Runtime behavior monitoring — tools like Falco watch all syscalls made by all containers in real-time and alert on suspicious behavior patterns. Falco rules describe normal and abnormal behavior: it's normal for a model server to make network connections on port 8080, read model files, and write logs. It's suspicious for a model server to spawn a shell process, download files with curl, modify system files, or make connections to unexpected external IPs.

Falco alerts in real time when these rules are violated:

yaml

6 lines

```
- rule: Shell spawned in container
  desc: A shell was spawned inside a container - potential exploitation
  condition: container.id != "" and proc.name in (shell_binaries)
  output: "Shell spawned (container=%container.name user=%user.name proc=%proc.name parent=%proc.pname)"
  priority: WARNING
```

This rule fires whenever a shell (bash, sh, zsh) is spawned inside any container. For production model servers built on distroless images, this should never happen legitimately — any alert is a genuine security event.

Immutable containers — in production, containers should be treated as immutable. The application code, dependencies, and configuration were baked into the image at build time. Nothing should change at runtime. Enforcing this through `readOnlyRootFilesystem: true` in the pod security context means even if an attacker gets code execution, they can't modify the container's filesystem to install persistence mechanisms. Combine this with network egress restrictions and you dramatically limit what an attacker can do with code execution in a container.

Resource limits as security — as mentioned in the Kubernetes section, resource limits are security controls, not just operational concerns. A container without CPU limits can perform a denial-of-service attack against other containers on the same node by consuming all available CPU. Memory without limits can OOM the entire node. Always set limits on production containers.

Audit container activity — beyond Falco's real-time alerting, audit logging of container activity helps with post-incident forensics. Knowing which files were accessed, which network connections were made, and which processes were spawned during a container's lifetime is essential for understanding the scope of a security incident. eBPF-based tools like Tetragon provide deep observability into container behavior with minimal overhead.

Regular image updates — runtime security also means keeping images updated. Even if your image had no known vulnerabilities when you built it, new CVEs are published constantly. Establish a process for regularly rebuilding images with updated base images and dependencies, scanning the rebuilt images, and rolling them out to production. For production ML systems, this might be monthly updates for baseline security maintenance and immediate updates when critical vulnerabilities are announced.

The connecting thread across all of them is the principle of defense in depth — security at every layer rather than relying on any single control. Build minimal images to reduce attack surface. Run as non-root to limit privilege. Scan for known vulnerabilities to catch the obvious. Sign images to ensure integrity. Monitor runtime behavior to catch the unexpected. No single control is sufficient, but together they create a layered security posture where each layer compensates for potential failures in others.

Kubernetes Production Concepts

01 Kubernetes Architecture Overview

Before diving into individual components, you need a mental model of what Kubernetes actually is and why it exists, because everything else makes more sense with that foundation.

Imagine you're running a large restaurant chain with hundreds of locations. Each location needs staff, supplies, and coordination. You could manage each location manually — calling each one, making sure they have what they need, replacing staff who don't show up. But that doesn't scale. You'd hire a regional management system that automatically ensures each location is staffed, restocked, and operating correctly — and if something goes wrong at one location, the system responds without you having to intervene personally.

Kubernetes is that regional management system, but for containerized applications. You have applications packaged as containers, and you need to run them reliably across many machines, scale them based on demand, replace them when they crash, and update them without downtime. Kubernetes automates all of this.

The fundamental model of Kubernetes is declarative. You tell Kubernetes what you want — "I need 5 replicas of this model serving container, each with 2 CPUs and 8GB of RAM, accessible on port 8080" — and Kubernetes figures out how to make that happen and keeps making it happen continuously. If a machine dies and takes 2 of your replicas with it, Kubernetes notices and starts 2 new replicas on healthy machines without you doing anything.

Kubernetes is a cluster — a group of machines (called nodes) that work together as a single system. These machines are split into two roles: the control plane and worker nodes. The control plane is the brain — it makes decisions, maintains the desired state, and orchestrates everything. Worker nodes are the muscle — they actually run your application containers.

Everything in Kubernetes is represented as an object — a structured record stored in the control plane's database. A Pod is an object. A Deployment is an object. A Service is an object. You interact with Kubernetes by creating, modifying, and deleting these objects, and Kubernetes continuously works to make the real world match the objects you've declared.

The API server is the single entry point for all interactions with Kubernetes. Whether you're using `kubectl` from a terminal, a CI/CD pipeline deploying an application, or an internal Kubernetes component checking the state of the cluster — everything goes through the API server. This centralization is what makes Kubernetes's consistent, auditable, and securable.

For ML systems specifically, Kubernetes provides the foundation for running training jobs that need GPU resources, serving models at scale with automatic scaling, managing the many microservices that make up an ML platform, and isolating different teams' workloads from each other. Understanding Kubernetes at a production level is non-negotiable for anyone working in MLOps or platform engineering.

02 Control Plane Components

The control plane is the collection of processes that make up Kubernetes's brain. It runs on dedicated nodes (separate from where your application workloads run in production setups) and is responsible for maintaining the desired state of the entire cluster. If the control plane goes down, your running workloads continue running, but you lose the ability to make changes, and Kubernetes stops automatically healing failures.

Let's go through each control plane component and understand exactly what it does.

kube-apiserver is the front door to everything. Every interaction with the cluster — from `kubectl` commands, from internal components talking to each other, from your CI/CD pipeline deploying a new model — goes through the API server. It's a RESTful HTTP API. When you run `kubectl apply -f deployment.yaml`, `kubectl` sends an HTTP request to the API server with your deployment object. The API server validates the request (is this syntactically valid? does this user have permission to do this?), persists the object to `etcd`, and returns a response. The API server is stateless — it doesn't store anything itself. It reads from and writes to `etcd` for all persistent data. In production, you run multiple replicas of the API server behind a load balancer for high availability.

etcd is the cluster's database — a distributed, consistent key-value store where all cluster state is persisted. Every Kubernetes object you create — every Pod, Deployment, ConfigMap, Secret, Service — is stored in `etcd`. It's the source of truth for the entire cluster. If `etcd` is lost and you have no backup, your cluster state is gone. You know what you're running from memory, but Kubernetes doesn't. This is why backing up `etcd` is critical in production. `etcd` uses the Raft consensus algorithm to ensure consistency across multiple `etcd` nodes — writes are only confirmed after a majority of `etcd` nodes have acknowledged them. In production, you run `etcd` as a cluster of 3 or 5 nodes to tolerate node failures while maintaining consistency.

kube-scheduler is responsible for deciding which worker node each new Pod should run on. When a new Pod is created and doesn't yet have a node assigned, the scheduler notices it, evaluates all available nodes against the Pod's requirements, and assigns it to the most suitable node. The scheduler considers many factors: does the node have enough CPU and memory (resource requests)? Does the Pod have node affinity rules specifying it must or should run on certain nodes? Does the Pod have anti-affinity rules saying it shouldn't run on the same node as other specific Pods (important for spreading replicas across nodes for fault tolerance)? Does the Pod require a GPU and does the node have one? The scheduler writes its decision back to the API server (updating the Pod object with the assigned node), and then the `kubelet` on that node picks up the Pod and starts running it.

kube-controller-manager runs a collection of control loops, each of which watches the state of some type of Kubernetes object and takes action to reconcile the current state with the desired state. The Deployment controller watches Deployment objects and ensures the correct number of ReplicaSets and Pods exist. The ReplicaSet controller watches ReplicaSet objects and ensures the correct number of Pods are running. The Node controller watches Node objects and marks nodes as unhealthy if they stop reporting in. The Job controller manages batch Jobs. There are dozens of built-in controllers. Each controller runs an independent loop: observe current state, compare to desired state, take action to close the gap, repeat. This reconciliation loop model is the core pattern of Kubernetes and understanding it conceptually unlocks almost everything else.

cloud-controller-manager integrates Kubernetes with the underlying cloud provider. When you create a Service of type LoadBalancer on AWS, something needs to actually call the AWS API to create an Elastic Load Balancer. That's the cloud controller manager. It handles cloud-specific operations: provisioning load

balancers, managing cloud storage volumes, updating cloud routing tables when nodes join or leave the cluster. It runs as a separate component from kube-controller-manager because each cloud provider implements it differently.

03 Worker Node Components

Worker nodes are the machines that actually run your application containers. Each worker node runs a set of components that enable it to receive work from the control plane, run containers, report status back, and provide networking.

kubelet is the primary agent on every worker node. It's the link between the control plane and the actual container runtime on the node. The kubelet constantly watches the API server for Pods that have been scheduled to its node. When a new Pod is assigned to the node, the kubelet reads the Pod specification and instructs the container runtime to start the containers described in the spec. Once containers are running, the kubelet continuously monitors their health, runs liveness and readiness probes (more on these later), and reports back to the API server. If a container crashes, the kubelet restarts it according to the Pod's restart policy. If the kubelet itself stops communicating with the API server (node failure, network partition), the node controller in the control plane marks the node as NotReady and eventually evicts its Pods to be rescheduled elsewhere.

Container Runtime is the software that actually pulls container images and runs containers. Kubernetes doesn't run containers directly — it delegates this to a container runtime via the Container Runtime Interface (CRI). containerd is the most common container runtime in modern Kubernetes clusters, and it's what most managed Kubernetes services (EKS, GKE, AKS) use. Docker used to be the dominant runtime but was deprecated as a Kubernetes runtime (not as a build tool — Docker is still fine for building images). The container runtime pulls images from registries, creates container file systems, manages container namespaces and cgroups, and handles container lifecycle operations.

kube-proxy runs on every node and maintains network rules that allow Pods to communicate with Services. When you create a Kubernetes Service, kube-proxy is responsible for ensuring that traffic sent to the Service's IP address gets forwarded to one of the backing Pods. It does this by maintaining iptables rules (or more modern alternatives like IPVS) on each node. When a Pod on node A makes a request to a Service, kube-proxy's rules on node A intercept the traffic and forward it to an appropriate backend Pod — which might be on the same node or a different node. In newer Kubernetes environments, eBPF-based CNI plugins like Cilium replace kube-proxy with a more efficient implementation.

Container Network Interface (CNI) Plugin handles Pod-to-Pod networking. Every Pod in a Kubernetes cluster gets its own IP address, and Pods on different nodes need to be able to communicate with each other directly using those IP addresses. The CNI plugin (Calico, Cilium, Flannel, Weave) implements the networking overlay or underlay that makes this possible. It creates network interfaces inside each Pod, assigns IP addresses, and sets up routing so that packets between Pods on different nodes get routed correctly. The choice of CNI plugin also determines which networking features are available — Calico and Cilium support NetworkPolicy for pod-level network isolation, while Flannel doesn't.

Node-level resource management — the kubelet also manages resources on the node. It enforces the CPU and memory limits you set on containers using Linux cgroups. It manages local ephemeral storage. It coordinates with the device plugin framework to expose specialized hardware to containers — this is how GPU resources are made available to ML training and inference containers.

04 Pod Lifecycle

A Pod is the smallest deployable unit in Kubernetes. Not a container — a Pod. A Pod is a group of one or more containers that share a network namespace (same IP address), the same storage volumes, and run on the same node. Containers within a Pod communicate with each other via localhost. In practice, most Pods have a single application container, sometimes with sidecar containers (like a service mesh proxy, a log shipper, or a metrics exporter) that provide auxiliary functionality.

Understanding the Pod lifecycle — the states a Pod goes through from creation to termination — is essential for debugging and for designing reliable systems.

Pending — the Pod has been created (the object exists in etcd) but hasn't been scheduled to a node yet, or has been scheduled but the containers haven't started yet. A Pod can stay in Pending for several reasons: the scheduler hasn't found a suitable node (not enough CPU/memory/GPU available), the container image is being pulled, or init containers are running. When debugging a stuck Pending Pod, `kubectl describe pod <name>` shows you the events and the reason it's not progressing.

Init Containers — before the main containers in a Pod start, you can define init containers that run to completion first. These are used for setup tasks: waiting for a database to be ready before starting the application, downloading configuration from an external service, initializing a shared volume. Init containers run sequentially — each must complete successfully before the next starts. Only after all init containers complete do the main containers start. This is useful for ML systems where the model server needs to download model weights before it can start serving.

Running — all containers in the Pod have been created and at least one is running. Running doesn't necessarily mean healthy — a container can be running but unable to serve traffic.

ContainerCreating — a sub-state of Pending/Running where the container runtime is pulling the image and setting up the container environment. This can take time for large images (ML model serving images are often several gigabytes).

Succeeded — all containers in the Pod have terminated successfully (exit code 0) and will not be restarted. This is the terminal state for batch job Pods.

Failed — all containers have terminated, and at least one container terminated with a failure (non-zero exit code or killed by the system). For Pods managed by Deployments, the controller creates a replacement Pod.

CrashLoopBackOff — this is one of the most common problematic states you'll see. It means a container is repeatedly crashing and Kubernetes keeps restarting it, but each time it crashes again. The "BackOff" part means Kubernetes adds increasing delays between restart attempts (10s, 20s, 40s, 80s, up to 5 minutes) to avoid hammering a broken container. The causes range from application errors (an exception on startup), misconfiguration (missing environment variable, wrong config file path), missing dependencies, insufficient resources, or invalid startup commands. `kubectl logs <pod>` and `kubectl logs <pod> --previous` (to see logs from the last crashed container) are your primary debugging tools.

Termination — when a Pod is deleted or evicted, it goes through a graceful termination process. Kubernetes sends a SIGTERM signal to all containers, giving them time to finish in-flight requests and clean up. The grace period is 30 seconds by default (configurable via `terminationGracePeriodSeconds`). During this period, the Pod is removed from Service endpoints so it no longer receives new traffic. After the grace period, if containers haven't exited, Kubernetes sends SIGKILL, forcefully terminating them. For ML serving containers, proper SIGTERM handling is important — you want to finish any in-flight prediction requests before shutting down.

Restart policies — `Always` restarts containers regardless of exit code (default for Deployment Pods, appropriate for long-running services). `OnFailure` only restarts on non-zero exit (appropriate for batch jobs). `Never` never restarts (for one-shot tasks).

05 Deployments & ReplicaSets

In practice, you almost never create Pods directly. You create Deployments, and Deployments manage Pods through an intermediate object called a ReplicaSet. Understanding this layering is important for understanding how updates and rollbacks work.

ReplicaSet is the object responsible for ensuring that a specified number of identical Pod replicas are running at any time. A ReplicaSet has a pod template (the specification for what the Pods should look like), a replica count (how many Pods should be running), and a selector (how it identifies which Pods it manages — typically using labels). The ReplicaSet controller continuously compares the number of running Pods matching its selector against its desired replica count. Too few Pods — create more. Too many — delete some. Pod crashes — create a replacement.

You could use ReplicaSets directly, but almost never should, because ReplicaSets don't support updates well. If you change the Pod template in a ReplicaSet (to update the container image version, for example), existing Pods are not automatically updated. You'd have to delete old Pods manually and let the ReplicaSet create new ones with the new template. This is error-prone.

Deployment is a higher-level abstraction that manages ReplicaSets to provide controlled, rolling updates and rollback capability. When you create a Deployment, it creates a ReplicaSet. When you update a Deployment (changing the container image, environment variables, or any part of the Pod template), the Deployment controller creates a new ReplicaSet with the updated Pod template and orchestrates a controlled transition from the old ReplicaSet to the new one.

A Deployment spec looks like this:

yaml

31 lines

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: model-server
  namespace: ml-production
spec:
  replicas: 5
  selector:
    matchLabels:
      app: model-server
  template:
    metadata:
      labels:
        app: model-server
    spec:
      containers:
        - name: model-server
```



```

    image: myregistry/model-server:v2.1.0
    resources:
      requests:
        memory: "8Gi"
        cpu: "2"
      limits:
        memory: "16Gi"
        cpu: "4"
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2          # Create up to 2 extra Pods during update
      maxUnavailable: 0    # Never reduce below 5 Pods during update

```

The `strategy` section controls how updates happen. `RollingUpdate` is the default and appropriate for most ML serving scenarios — it gradually replaces old Pods with new ones, maintaining service throughout the update. `Recreate` kills all old Pods before creating new ones, causing a brief downtime — only appropriate when you absolutely cannot run two versions simultaneously.

`maxSurge` controls how many extra Pods above the desired replica count can exist during an update. `maxUnavailable` controls how many Pods can be unavailable below the desired count. Setting `maxSurge: 2` and `maxUnavailable: 0` means during an update, Kubernetes first creates 2 new Pods with the new version (taking you to 7 total), waits for them to be ready, then deletes 2 old Pods (back to 5), and repeats until all old Pods are replaced. You always have at least 5 serving Pods throughout.

Deployment vs Pod directly — always use Deployments (or other controllers like StatefulSets or Jobs) rather than bare Pods. A bare Pod deleted is gone forever — nothing recreates it. A Deployment whose Pod is deleted immediately creates a replacement. Deployments give you declarative updates, rollback history, and the full suite of Kubernetes reliability features.

06 Services & Service Types

Pods are ephemeral. They come and go — they crash and get replaced, they get rescheduled to different nodes, they're replaced during updates. Each new Pod gets a new IP address. If your frontend needs to talk to your model server, it can't rely on a fixed Pod IP because that IP changes constantly.

Services solve this by providing a stable network endpoint that sits in front of a dynamic set of Pods. A Service has a fixed IP address (called a ClusterIP) and a fixed DNS name that never change, regardless of how many times the backing Pods are replaced. Traffic sent to the Service is load-balanced across the healthy Pods matching the Service's selector.

The selector on a Service uses the same label system as everything else in Kubernetes. If your model server Pods have the label `app: model-server`, a Service with selector `app: model-server` automatically directs traffic to all Pods with that label. When a new Pod is created with that label, it's automatically added to the Service's endpoint list. When a Pod is deleted, it's automatically removed. The Service tracks healthy backends dynamically without any manual configuration.

Kubernetes has four Service types, each appropriate for different networking scenarios.

ClusterIP is the default. It creates a virtual IP address accessible only from within the cluster. Services use this for internal communication — your API gateway talks to your model server via ClusterIP, your model server talks to your feature store via ClusterIP. This IP is not accessible from outside the cluster, which is correct for most backend services. Every ClusterIP Service also gets a DNS name in the format `service-name.namespace.svc.cluster.local`, so services can discover each other by name rather than IP.

NodePort exposes the Service on a static port (between 30000-32767) on every node in the cluster. Traffic sent to any node's IP address on that port is forwarded to the Service. This makes the Service accessible from outside the cluster by connecting to `<any-node-ip>:<nodeport>`. NodePort is rarely used in production because you're exposing a high-numbered port on every node, there's no load balancing across nodes, and you're relying on node IPs which can change. It's sometimes used for development or debugging.

LoadBalancer is the standard way to expose a Service to the internet or external network in cloud environments. When you create a LoadBalancer Service, the cloud-controller-manager calls the cloud provider's API (AWS, GCP, Azure) to provision an external load balancer (ELB, GCLB, Azure LB) and configures it to forward traffic to the NodePort on all cluster nodes, which then forwards to the appropriate Pods. You get a single, stable external IP or DNS name. This is how you'd expose a model serving API to external clients. The downside is cost — each LoadBalancer Service provisions a cloud load balancer, which costs money. For exposing many services, an Ingress controller is more cost-effective.

ExternalName maps a Service to an external DNS name rather than a Pod selector. This lets you refer to external services using Kubernetes's internal DNS system. Your model server can access a legacy external database at `Legacy-db` instead of a long external hostname, and the ExternalName Service handles the DNS alias. This is useful for migrating services gradually — you point the ExternalName at an external service, and when you bring that service into the cluster, you update the Service type without changing how other services reference it.

Ingress is not a Service type but is closely related. An Ingress is a set of rules for routing external HTTP/HTTPS traffic to Services within the cluster. An Ingress controller (nginx, Traefik, AWS ALB Ingress Controller) watches Ingress objects and configures itself accordingly. Ingress lets you route different URL paths to different Services, handle TLS termination, and expose many services through a single external load balancer. For ML platforms with many services (prediction API, training dashboard, monitoring, model registry UI), Ingress is the typical production approach.

07 ConfigMaps & Secrets

Applications need configuration — database hostnames, feature flags, model hyperparameters, API endpoints, log levels. This configuration should not be hardcoded into container images, because then every configuration change requires rebuilding and redeploying the image. ConfigMaps and Secrets are Kubernetes's mechanisms for injecting configuration into containers without embedding it in images.

ConfigMaps store non-sensitive configuration data as key-value pairs. A ConfigMap can store simple values (a single string for an environment variable) or entire configuration files (a YAML config file, a properties file, an nginx configuration). ConfigMaps are stored in etcd as plain text — they're not encrypted or treated with any special security.

Creating a ConfigMap:

yaml

15 lines

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: model-server-config
  namespace: ml-production
data:
  MODEL_VERSION: "v2.1.0"
  BATCH_SIZE: "32"
  LOG_LEVEL: "INFO"
  feature_store_config.yaml: |
    host: feature-store-service
    port: 6379
    connection_pool_size: 10
    timeout_seconds: 5
```

This ConfigMap has both simple key-value pairs and an entire YAML file as a value.

You consume ConfigMaps in Pods in two ways. As environment variables, where individual keys become environment variables in the container:

yaml

7 lines

```
env:
- name: MODEL_VERSION
  valueFrom:
    configMapKeyRef:
      name: model-server-config
      key: MODEL_VERSION
```

Or mounting the entire ConfigMap as files in a directory, where each key becomes a filename and the value becomes the file contents:

yaml

8 lines

```
volumeMounts:
- name: config-volume
  mountPath: /etc/model-server
volumes:
- name: config-volume
  configMap:
    name: model-server-config
```

After mounting, your container finds `/etc/model-server/feature_store_config.yaml` with the content you defined in the ConfigMap. This is how you inject complete configuration files into containers.

Secrets work identically to ConfigMaps in terms of how they're consumed (environment variables or volume mounts), but they're designed for sensitive data — passwords, API keys, TLS certificates, database credentials. Kubernetes applies some protections to Secrets that aren't applied to ConfigMaps: they're not displayed in `kubectl describe` output (values are masked), they're stored in tmpfs (memory) on nodes rather than disk, and they're only sent to nodes that have Pods requesting them.

yaml

10 lines

```
apiVersion: v1
kind: Secret
metadata:
  name: ml-database-credentials
  namespace: ml-production
type: Opaque
data:
  username: bWwtdXNlcg==      # base64-encoded "ml-user"
  password: c3VwZXJzZWNyZXQ= # base64-encoded "supersecret"
```

Important caveat: Kubernetes Secrets are base64-encoded, not encrypted. Base64 is encoding, not encryption — anyone with access to the Secret object can decode it trivially. Kubernetes Secrets are more about operational separation (distinguishing sensitive from non-sensitive config) than genuine security. For real security, you should either enable encryption at rest for Secrets in etcd (so the data is actually encrypted in the database) or use an external secrets management system like Vault with the External Secrets Operator, which keeps the actual secret values in Vault and only creates Kubernetes Secrets at runtime.

Automatic Secret injection — Kubernetes automatically injects a service account token Secret into every Pod, which the Pod's processes can use to authenticate to the Kubernetes API. This enables patterns where application code queries Kubernetes (checking pod count, reading ConfigMaps dynamically, etc.) without needing separately managed credentials.

08 Resource Requests & Limits

Resource management in Kubernetes is one of the most important and most often misconfigured aspects of running production workloads, especially ML workloads which are inherently resource-intensive.

Every container in a Pod can (and in production should) specify two things: how much CPU and memory it expects to use (requests), and the maximum CPU and memory it's allowed to use (limits).

Resource Requests are the amount of CPU and memory that the scheduler uses when deciding where to place a Pod. The scheduler finds nodes that have at least as much unallocated (not already requested by other Pods) CPU and memory as the Pod's requests. Requests don't mean the container is actually using that much — they're a reservation. If you request 4 CPUs and your container is idle, it's using almost no CPU, but the scheduler treats 4 CPUs as reserved on that node and won't place other Pods there if it would exceed the node's total capacity.

Resource Limits are the maximum amount of CPU and memory the container is allowed to use, enforced by Linux cgroups.

CPU limits are enforced through throttling — if a container tries to use more CPU than its limit, the kernel throttles it. The container continues running but is slowed down. CPU is measured in millicores — 1000m (millicores) equals 1 CPU core. `cpu: "500m"` means half a CPU core.

Memory limits are enforced through OOMKill — if a container tries to allocate more memory than its limit, the kernel's Out-of-Memory killer terminates the process with an OOMKilled exit code. The container is then restarted by the kubelet according to the restart policy. Memory is measured in bytes, with suffixes: Mi (mebibytes), Gi (gibibytes). `memory: "4Gi"` means 4 gibibytes.

yaml

8 lines

```
resources:
  requests:
    memory: "8Gi"
    cpu: "2"
  limits:
    memory: "16Gi"
    cpu: "4"
```

This container is guaranteed at least 2 CPUs and 8Gi of memory (the scheduler ensures this is available), and is allowed to burst up to 4 CPUs and 16Gi.

Setting requests and limits correctly is nuanced and critically important for ML workloads.

If requests are too low, the scheduler will overcommit nodes — placing many Pods on a node whose combined requests fit, but whose combined actual usage exceeds the node's capacity. This causes performance problems and OOMKills when actual usage spikes.

If requests are too high (over-requesting), Pods won't be scheduled efficiently — nodes appear full to the scheduler even when actual resource usage is low, wasting money on underutilized nodes.

If memory limits are too low, your model server will be OOMKilled mid-inference, causing failed predictions and poor user experience. This is a common problem with ML containers where memory usage can be variable depending on input size.

If you don't set limits at all, a single misbehaving container can consume all resources on a node, starving other containers. In production, always set limits.

GPU resources — for ML training and inference containers, you also request GPU resources:

yaml

4 lines

```
resources:
  limits:
    nvidia.com/gpu: 2 # Request 2 GPUs
```

GPU resources are always specified as limits only (requests equal limits automatically). GPUs are not shared between containers by default — one Pod gets exclusive access to the requested GPUs.

LimitRange is a namespace-level object that sets default requests and limits for Pods that don't specify them, and enforces minimum and maximum values. This prevents anyone from deploying a container with no limits (which could monopolize node resources) or requesting more resources than is reasonable.

ResourceQuota sets aggregate resource limits for an entire namespace — the total CPU, memory, and GPU that all Pods in the namespace combined can request. This is how you implement multi-tenancy resource allocation — give the ML training namespace a budget of 100 CPUs and 400Gi of memory, and Kubernetes enforces that no more than that is requested across all Pods in that namespace.

09 Liveness & Readiness Probes

Kubernetes can automatically detect that a container's process has crashed (the process exits), but it can't automatically detect more subtle failures — a web server that's running but stuck in a deadlock and not responding to requests, a model server that's loaded but still initializing and not ready to serve traffic yet, or an application that has run out of a connection pool and is returning errors. Probes are how you teach Kubernetes to detect these conditions.

There are three types of probes, each serving a distinct purpose.

Liveness Probes answer the question: is this container still alive and functional? If a liveness probe fails, Kubernetes considers the container to be broken and kills and restarts it. This handles scenarios where the process is still running but is in an unrecoverable bad state — hung, deadlocked, or stuck in an infinite error loop.

A liveness probe for an HTTP server checks a specific endpoint:

yaml

9 lines

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30    # Wait 30s after container starts before probing
  periodSeconds: 10         # Check every 10 seconds
  failureThreshold: 3       # Kill and restart after 3 consecutive failures
  timeoutSeconds: 5         # Probe times out after 5 seconds
```

The `initialDelaySeconds` is critical — without it, Kubernetes might try to probe your container before it has finished starting up and immediately kill it, causing a crash loop before the application even has a chance to start. For ML model servers that need to load large model weights, this might need to be 60, 120, or even more seconds.

Readiness Probes answer the question: is this container ready to receive traffic? A container that fails its readiness probe is removed from the Service's endpoint list — traffic stops being routed to it — but the container is not killed or restarted. This handles the startup phase (container is running but still loading model weights, still warming up caches, still connecting to dependencies) and temporary overload (container is too busy to handle more traffic).

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 3
  successThreshold: 1 # Must pass once to be considered ready again
```

The difference between liveness and readiness is subtle but important. Liveness failure = restart. Readiness failure = stop sending traffic. A container can be live (not dead, no need to restart) but not ready (temporarily unable to serve). A common pattern for model servers: the readiness probe checks that the model is loaded and the server can handle requests, while the liveness probe checks that the server process itself is not hung.

Startup Probes are the third type and solve a specific problem — applications that have very long initialization times. If your liveness probe runs during startup and the application takes 3 minutes to load models, the liveness probe would repeatedly fail and restart the container before it ever gets a chance to finish starting. You could set `initialDelaySeconds: 180` on the liveness probe, but then you'd have a 3-minute detection gap for actual failures after startup.

Startup probes solve this cleanly. The startup probe runs from container start and disables both liveness and readiness probes until the startup probe succeeds. Once the startup probe succeeds (application has finished initializing), normal liveness and readiness probes take over. This gives slow-starting applications all the time they need to start without compromising health detection once they're running.

```
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 30 # Try 30 times
  periodSeconds: 10 # Every 10 seconds = up to 300 seconds (5 minutes) to start
```

Probe types — probes are not limited to HTTP. You can also use TCP probes (checks that a TCP connection can be established on a port — useful for non-HTTP services) and exec probes (runs a command inside the container and checks the exit code — flexible but more resource-intensive).

10 Horizontal Pod Autoscaler (HPA)

Manually setting replica counts for your Deployments is appropriate for stable, predictable workloads. But ML serving workloads are rarely stable and predictable. Traffic has daily patterns (low at night, high during business hours), viral moments can cause sudden 10× traffic spikes, and batch inference jobs can suddenly increase load. The Horizontal Pod Autoscaler automatically adjusts the number of Pods in a Deployment based on observed metrics.

The HPA is a control loop. It periodically (every 15 seconds by default) queries the metrics server for current resource usage of the Pods it's managing, compares that to the target you've specified, calculates the desired replica count, and updates the Deployment if the current count differs.

The basic form of HPA scales on CPU utilization:

yaml

19 lines

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: model-server-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: model-server
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70 # Scale to maintain 70% CPU utilization average
```

This HPA maintains between 3 and 20 replicas of the model-server Deployment, scaling up when average CPU utilization across all Pods exceeds 70% and scaling down when it drops significantly below 70%. The formula Kubernetes uses is: $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times (\text{currentMetric} / \text{targetMetric}))$. If you have 5 replicas at 140% CPU utilization and target is 70%, $\text{desired} = \text{ceil}(5 \times (140/70)) = \text{ceil}(10) = 10$ replicas.

Custom metrics — for ML systems, CPU is often not the best scaling signal. A model server might be CPU-idle but GPU-saturated. Or it might be better to scale on inference request latency or queue depth. The HPA supports custom metrics via adapters like the Prometheus Adapter or KEDA.

Scaling on requests per second using Prometheus metrics:

```
metrics:
- type: Pods
  pods:
    metric:
      name: http_requests_per_second
    target:
      type: AverageValue
      averageValue: "100" # Target 100 requests per second per Pod
```

KEDA (Kubernetes Event-Driven Autoscaling) extends HPA with many more scaling sources — Kafka topic lag, SQS queue depth, Redis list length, Prometheus metrics. For ML systems where inference requests come through a message queue, scaling on queue depth is much more effective than scaling on CPU. More messages in the queue means more workers needed — straightforward and responsive.

Scale-down stabilization — Kubernetes is conservative about scaling down to prevent flapping (rapid scale-up and scale-down cycles). By default, scale-down decisions require 5 minutes of sustained low load before pods are removed. This window is configurable. For ML serving, you generally want fast scale-up (add capacity immediately when traffic spikes) and slow scale-down (don't remove pods until you're confident traffic has sustainably dropped).

Resource requests are required — HPA based on CPU utilization only works if your containers have resource requests defined. The utilization percentage is calculated relative to the requested CPU. Without requests, the metrics server can't calculate utilization percentage, and HPA won't function.

11 Rolling Updates & Rollbacks

Updates to production services are a constant reality — new model versions, bug fixes, configuration changes, dependency updates. Kubernetes provides built-in mechanisms for updating Deployments with zero downtime and quickly reverting if something goes wrong.

Rolling Updates are the default update strategy for Deployments. When you update a Deployment (change the container image tag, modify an environment variable, update resource limits), Kubernetes doesn't kill all old Pods and start new ones simultaneously. It gradually replaces old Pods with new ones, ensuring that the service remains available throughout the update.

The process: Kubernetes creates a new ReplicaSet with the updated Pod template. It starts creating new Pods from the new ReplicaSet. As new Pods become Ready (pass their readiness probes), old Pods are terminated from the old ReplicaSet. This continues until all Pods are running the new version.

The speed and safety of the rolling update is controlled by `maxSurge` and `maxUnavailable` in the Deployment strategy (covered in the Deployments section). For ML serving, the typical production configuration is `maxSurge: 25%` and `maxUnavailable: 0` — this ensures you always have full capacity, at the cost of needing up to 25% extra capacity during the update.

Triggering an update — the most common trigger is updating the container image:

bash

2 lines

```
kubectl set image deployment/model-server model-server=myregistry/model-server:v2.2.0
```

Or editing the Deployment manifest and applying it:

bash

2 lines

```
kubectl apply -f deployment.yaml
```

Monitoring the rollout — Kubernetes provides a command to watch the rollout progress:

bash

2 lines

```
kubectl rollout status deployment/model-server
```

This blocks and shows progress until the rollout completes. In CI/CD pipelines, this command is used to verify that a deployment actually succeeded before marking the pipeline as successful.

Rollout history — Kubernetes keeps a history of Deployment revisions. You can view the history:

bash

2 lines

```
kubectl rollout history deployment/model-server
```

This shows each revision with the change cause (if you annotate your deployments with change descriptions using `--record` or the kubernetes.io/change-cause annotation).

Rollbacks — if a new version is causing problems, rolling back is a single command:

bash

2 lines

```
kubectl rollout undo deployment/model-server
```

This reverts to the previous revision. To revert to a specific revision:

bash

2 lines

```
kubectl rollout undo deployment/model-server --to-revision=3
```

Under the hood, a rollback is just a new rolling update — Kubernetes updates the Deployment's Pod template back to the previous version and performs a new rolling update. The old ReplicaSet (which Kubernetes kept around for exactly this reason) is scaled back up while the current ReplicaSet is scaled down.

The key to safe rollouts — readiness probes. During a rolling update, Kubernetes only considers a new Pod ready (and starts terminating an old Pod) when the readiness probe passes. If your new model version fails its readiness probe — because it crashes on startup, because the model weights are wrong, because there's

a configuration error — Kubernetes stops the rollout and leaves your old Pods running. The rollout hangs rather than replacing working Pods with broken ones. This is the primary safety mechanism — without readiness probes, Kubernetes would replace working Pods with broken ones regardless.

12 Namespaces & Multi-Tenancy

A Kubernetes cluster is a shared resource. Multiple teams, multiple applications, multiple environments might all share the same cluster. Without mechanisms for isolation and organization, this shared resource becomes chaotic — one team's misbehaving application affects another's, there are no resource boundaries, and permissions are a mess.

Namespaces are Kubernetes's primary mechanism for organizing a cluster into virtual partitions. Each namespace is a scope for resources — Pods, Deployments, Services, ConfigMaps, Secrets — that provides naming isolation, access control boundaries, and resource quota enforcement.

The key properties of namespaces:

Naming isolation — resource names must be unique within a namespace but can be duplicated across namespaces. You can have a Deployment named `model-server` in both the `ml-team-a` namespace and `ml-team-b` namespace simultaneously. Each team manages their own resources without naming conflicts.

Access control — RBAC permissions are scoped to namespaces. You can give Team A's engineers admin access to the `ml-team-a` namespace without giving them any access to `ml-team-b`. Service accounts in one namespace don't automatically have access to resources in another. This is how you implement multi-tenancy where different teams can use the same cluster without being able to interfere with each other.

Resource quotas — you can set ResourceQuota objects on namespaces to limit the total CPU, memory, GPU, and number of objects that can be created in that namespace. Give the training namespace a budget of 200 CPUs and 800Gi of memory. Once that budget is consumed, new Pods won't be scheduled until existing ones are deleted. This prevents one team from monopolizing shared cluster resources.

Network isolation — combined with NetworkPolicy objects, namespaces provide a unit for network segmentation. You can define policies that allow traffic within a namespace but restrict traffic between namespaces, or that allow specific cross-namespace communication for well-defined integration points.

Typical namespace organization patterns:

By environment: `development`, `staging`, `production` namespaces. Each environment gets resource quotas appropriate to its scale. This works well for smaller organizations but has the downside that a noisy staging workload can affect production if they're in the same cluster.

By team: `ml-platform`, `data-science-team-a`, `data-science-team-b`, `monitoring`, `infrastructure`. Each team manages their namespace independently. Resource quotas enforce fair sharing.

By function: `model-training`, `model-serving`, `feature-store`, `monitoring`, `data-pipelines`. Different components of your ML platform in different namespaces with appropriate resource allocations for each function.

Hybrid approaches are common — `production-model-serving`, `production-model-training`, `staging-ml` — combining environment and function for clear separation.

Default Kubernetes namespaces — clusters come with built-in namespaces: `default` (where resources land if you don't specify a namespace — avoid using this in production), `kube-system` (Kubernetes system components), `kube-public` (publicly readable cluster info), `kube-node-lease` (node heartbeat objects). Don't put your workloads in `kube-system` or `default`.

13 RBAC & Access Control

We covered RBAC conceptually in the Security section, but here we'll go deeper into the Kubernetes-specific mechanics because understanding these mechanics is essential for production cluster operation.

RBAC in Kubernetes has four object types that combine to define who can do what.

Role defines a set of permissions within a specific namespace. It contains rules that each specify: which API groups, which resources, and which verbs (actions) are allowed.

yaml

16 lines

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: model-deployer
  namespace: ml-production
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
- apiGroups: [""]
  resources: ["pods", "pods/logs"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list", "watch"]
```

This Role allows getting, listing, watching, creating, and updating Deployments in the `ml-production` namespace, and reading Pods and ConfigMaps. It cannot delete Deployments, cannot access Secrets, and cannot do anything in other namespaces.

ClusterRole works like Role but applies cluster-wide across all namespaces, or for cluster-scoped resources (Nodes, PersistentVolumes, Namespaces themselves) that don't belong to any namespace. Use ClusterRole for permissions that genuinely need cluster-wide scope, and namespace-scoped Roles for everything else.

RoleBinding grants a Role to a subject (user, group, or service account) within a specific namespace:

yaml

17 lines

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ml-engineer-model-deployer
  namespace: ml-production
subjects:
- kind: User
  name: "harsh@company.com"
  apiGroup: rbac.authorization.k8s.io
- kind: Group
  name: "ml-engineers"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: model-deployer
  apiGroup: rbac.authorization.k8s.io
```

ClusterRoleBinding grants a ClusterRole to subjects cluster-wide. Use sparingly — most permissions should be namespace-scoped.

Service Accounts are the identity mechanism for Pods. Every Pod runs as a service account. By default, Pods run as the `default` service account in their namespace, which has minimal permissions. You create dedicated service accounts for Pods that need specific Kubernetes API access:

yaml

6 lines

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: model-server-sa
  namespace: ml-production
```

Bind a Role to this service account:

yaml

5 lines

```
subjects:
- kind: ServiceAccount
  name: model-server-sa
  namespace: ml-production
```

And specify it in the Pod spec:

```
spec:
  serviceAccountName: model-server-sa
```

Common RBAC mistakes to avoid:

Using the `cluster-admin` ClusterRole for anything other than absolute necessity — it gives unrestricted access to everything and bypasses all RBAC checks.

Binding ClusterRoles using ClusterRoleBindings when RoleBindings would suffice — giving namespace-scoped permissions via ClusterRoleBinding accidentally grants access to all namespaces.

Giving service accounts of application Pods broad permissions — a model serving Pod almost certainly needs zero Kubernetes API access. If it does need some (reading its own ConfigMaps dynamically, for example), scope it precisely.

Not auditing RBAC regularly — permissions accumulate. That service account created for a one-time migration task six months ago still has broad permissions and nobody remembers what it does.

14 Persistent Volumes & Storage Classes

Containers are ephemeral by design — when a container restarts, its local filesystem is reset to the image state. Any data written inside the container during its lifetime is lost. For most stateless application containers, this is fine and even desirable. But for ML workloads — training data, model checkpoints, experiment logs, feature data — you need storage that persists independently of container lifecycle.

Kubernetes has a three-layer storage abstraction: PersistentVolumes, PersistentVolumeClaims, and StorageClasses.

PersistentVolume (PV) represents an actual storage resource in the cluster — an EBS volume on AWS, a Persistent Disk on GCP, an NFS share, a local disk on a node. PVs exist independently of Pods. A PV provisioned from an AWS EBS volume persists even when all Pods using it are deleted. PVs have a capacity, access mode, and reclaim policy.

Access modes define how the volume can be mounted: - `ReadWriteOnce (RWO)` — can be mounted read-write by a single node at a time. Most block storage (EBS, GCE PD) is RWO. You can have multiple Pods on the same node reading and writing, but not Pods on different nodes. - `ReadOnlyMany (ROX)` — can be mounted read-only by many nodes simultaneously. Useful for distributing static data (like pre-trained model weights) to many Pods. - `ReadWriteMany (RWX)` — can be mounted read-write by many nodes simultaneously. Requires network-attached storage (NFS, EFS, CephFS). Important for ML training jobs that need multiple Pods to write to shared storage.

PersistentVolumeClaim (PVC) is a request for storage by a user or application. A Pod doesn't reference a PV directly — it references a PVC, and Kubernetes finds a PV that satisfies the claim and binds them together. This decouples the Pod specification from the specific storage implementation. Your Pod just says "I need 50Gi of ReadWriteOnce storage" — whether that comes from an EBS volume or a GCE Persistent Disk is an infrastructure detail the Pod doesn't need to know.

yaml

13 lines

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: model-checkpoint-storage
  namespace: ml-training
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Gi
  storageClassName: gp3-encrypted
```

And the Pod references the PVC:

yaml

10 lines

```
volumes:
- name: checkpoints
  persistentVolumeClaim:
    claimName: model-checkpoint-storage
containers:
- name: training-job
  volumeMounts:
    - name: checkpoints
      mountPath: /checkpoints
```

StorageClass defines how PVs are dynamically provisioned. Instead of manually creating EBS volumes and PV objects before every training job, StorageClass enables dynamic provisioning — when a PVC is created, Kubernetes automatically provisions the underlying storage and creates the PV. You define a StorageClass with the provisioner (the component that creates storage), parameters (volume type, encryption, IOPS), and reclaim policy (what happens to the PV when the PVC is deleted — Delete to destroy the storage, or Retain to keep it).

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-encrypted
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
  iops: "3000"
  throughput: "125"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

`WaitForFirstConsumer` means the volume is only provisioned when a Pod actually claims it, ensuring it's provisioned in the same availability zone as the Pod (critical for EBS which is AZ-local).

For ML workloads, storage is a first-class concern. Training checkpoints need durable, high-throughput storage. Model artifacts need persistent storage accessible by serving infrastructure. Feature data might need ReadWriteMany storage accessible by multiple training jobs in parallel. Shared dataset storage for distributed training. Understanding what access modes and performance characteristics each workload needs and selecting the appropriate StorageClass is part of production ML infrastructure design.

15 StatefulSets vs Deployments

Deployments are the right abstraction for stateless applications — your model serving containers, API gateways, and most microservices. But some components of your ML infrastructure are stateful — they have identity, stable storage, and ordered startup/shutdown requirements. For these, StatefulSets are the appropriate abstraction.

Understanding the difference between Deployments and StatefulSets comes down to understanding the difference between stateless and stateful workloads.

In a Deployment, all Pods are interchangeable. If Pod A dies, Pod B (its replacement) is functionally identical and clients don't care which one they're talking to. Pods don't have persistent individual identities. You can scale a Deployment up and down and any Pod can handle any request.

StatefulSets provide three things that Deployments don't:

Stable, predictable Pod names — Pods in a StatefulSet are named `<statefulset-name>-0`, `<statefulset-name>-1`, `<statefulset-name>-2`, etc. These names are stable — if Pod `ml-feature-store-2` is deleted, its replacement is also named `ml-feature-store-2`, not a random generated name. This is important for systems where identity matters — like distributed databases where each node has a specific role or partition assignment.

Stable network identity — each StatefulSet Pod gets a stable DNS hostname: `<pod-name>.<statefulset-name>.<namespace>.svc.cluster.local`. This hostname is persistent even when the Pod is rescheduled to a different node. Cluster members can address each other by predictable hostnames rather than ephemeral IPs.

Ordered, controlled startup and shutdown — StatefulSet Pods are created in order: Pod 0 must be Running and Ready before Pod 1 starts, Pod 1 before Pod 2, and so on. They're also deleted in reverse order. This ordering is essential for distributed systems that have bootstrap sequences — the first node initializes the cluster, subsequent nodes join it. Disordered startup would leave the cluster in an undefined state.

Persistent storage per Pod — StatefulSets support volume claim templates, which automatically create a dedicated PVC for each Pod. Pod 0 gets its own PVC, Pod 1 gets its own, and so on. When a Pod is rescheduled, it reconnects to its own PVC — the same storage it had before. This is how distributed databases maintain their data across Pod restarts.

When to use StatefulSets: - Databases (PostgreSQL, MySQL, MongoDB, Cassandra) - Distributed coordination services (ZooKeeper, etcd when running as application) - Message brokers (Kafka, RabbitMQ) - Feature stores with stateful backends (Redis cluster, Cassandra-backed feature stores) - Any application where each instance has unique identity, stable storage, or ordered lifecycle requirements

When to use Deployments: - Model serving containers (stateless — any replica can serve any request) - API gateways - Web servers - Data processing workers (stateless — any worker can process any job) - Most microservices

A common mistake is reaching for StatefulSets when Deployments would suffice, because it seems like the "more serious" choice. StatefulSets are significantly more complex to operate — rolling updates are slower (sequential rather than parallel), debugging requires understanding Pod ordinals, and scaling has operational implications. Use Deployments by default and only switch to StatefulSets when you genuinely need what they provide.

16 Kubernetes Networking Basics

Networking in Kubernetes is a topic that confuses many engineers because there are multiple layers of networking happening simultaneously, each solving a different problem. Let's build up the mental model layer by layer.

The fundamental networking model — Kubernetes has a flat networking model with three rules: every Pod gets a unique IP address, every Pod can communicate directly with every other Pod in the cluster without NAT (no address translation, the original source IP is preserved), and Pods and nodes can communicate directly. This is a remarkably simple model for what could be a very complex problem across many machines.

The flat model is implemented by the CNI plugin (Calico, Cilium, Flannel). The CNI plugin creates an overlay network (a virtual network on top of the physical network) or configures the physical network routing so that Pod IPs are routable across nodes. The specifics differ between CNI plugins, but the result is the same: Pod A on Node 1 can send a packet to Pod B on Node 3 using Pod B's IP address directly.

Pod networking — each Pod gets a private network namespace. The Pod's containers share this namespace — they all have the same IP address and can communicate with each other via localhost. The Pod's IP is assigned from the cluster's Pod CIDR (a range of IP addresses designated for Pods, typically something like `10.244.0.0/16`).

Service networking — Services have a separate IP range (ClusterIP range, typically `10.96.0.0/12`). Service IPs are virtual — no machine actually has that IP address. When a Pod sends a packet to a Service IP, kube-proxy's iptables rules on the local node intercept the packet, select a backend Pod from the Service's

endpoint list, and rewrite the destination IP to the chosen Pod's real IP before forwarding. This NAT happens transparently, and the reverse NAT happens on the return path.

DNS — CoreDNS runs as a Deployment in the `kube-system` namespace and provides DNS for the cluster. Every Service gets a DNS record: `<service-name>.<namespace>.svc.cluster.local`. Pods are configured to use CoreDNS as their DNS resolver, so `curl http://feature-store.ml-platform.svc.cluster.local` from any Pod resolves to the feature-store Service's ClusterIP. Within the same namespace, you can use just the service name: `curl http://feature-store`.

Ingress networking — traffic coming from outside the cluster enters through an Ingress controller (or a LoadBalancer Service). The Ingress controller is typically a reverse proxy (nginx, Envoy, Traefik) running as a Pod with a Service of type LoadBalancer. External traffic hits the load balancer (provisioned in the cloud), which forwards to the Ingress controller Pod, which inspects the HTTP host and path, matches it against Ingress rules, and forwards to the appropriate backend Service.

Network Policies — by default, all Pods can communicate with all other Pods. NetworkPolicy objects restrict this. A NetworkPolicy selects Pods by label and defines ingress rules (who can send traffic to the selected Pods) and egress rules (where the selected Pods can send traffic). NetworkPolicies are additive — if no NetworkPolicy selects a Pod, all traffic is allowed. Once any NetworkPolicy selects a Pod, only traffic explicitly allowed by NetworkPolicies is permitted.

The typical production pattern is a default-deny NetworkPolicy in each namespace that blocks all ingress and egress, followed by specific allow policies for required communication paths. Your model server allows ingress from the API gateway namespace on port 8080, and egress to the feature store on port 6379. Everything else is blocked.

DNS resolution troubleshooting — a very common networking issue in Kubernetes is DNS resolution failures. Debug using: `kubectl run test --image=busybox --rm -it -- nslookup feature-store.ml-platform.svc.cluster.local`. This creates a temporary Pod and runs a DNS lookup from inside the cluster, confirming whether the Service DNS record resolves correctly.

Port forwarding for debugging — when you need to access a service from your local machine without exposing it externally: `kubectl port-forward service/model-server 8080:80`. This tunnels traffic from your localhost port 8080 to the cluster Service's port 80, letting you test the service from your laptop without creating an external LoadBalancer.

Headless Services — a Service with `clusterIP: None` doesn't get a virtual IP. Instead, DNS queries for the Service return the individual Pod IPs directly. This is how StatefulSet Pods get their stable DNS names — each Pod gets its own DNS record via a headless Service. Applications that need to connect to specific individual backends (like a Cassandra driver connecting to specific Cassandra nodes) use headless Services to discover and connect to individual Pods by name.

The unifying idea across all of them is the reconciliation loop — Kubernetes is continuously watching the difference between what you declared and what actually exists, and constantly working to close that gap. Every component, from the scheduler to the HPA to the Deployment controller, is a specialized reconciliation loop watching some part of the cluster state and taking action to make reality match your declarations. Once you internalize this mental model, Kubernetes stops being a collection of confusing objects and becomes a coherent system with a clear purpose.

Observability & Monitoring

01 Observability Principles

Before anything else, let's separate two concepts that are often conflated: monitoring and observability. They're related but fundamentally different in philosophy, and understanding the difference shapes how you build systems.

Monitoring is the practice of collecting predefined metrics and checking them against predefined thresholds. You know in advance what might go wrong, you instrument for it, and you alert when it happens. CPU above 90%? Alert. Error rate above 1%? Alert. Disk full? Alert. Monitoring answers questions you thought to ask before the incident.

Observability is the property of a system that allows you to understand its internal state by examining its external outputs. An observable system lets you ask arbitrary questions about what's happening inside it — questions you didn't think to ask before, questions that arise during an incident when you're trying to understand something you've never seen before. Observability answers questions you didn't know you'd need to ask.

The term comes from control theory. A system is observable if you can determine its internal state from its outputs. Applied to software: an observable software system is one where, when something unexpected happens, you have enough data to understand why.

The practical difference matters enormously during incidents. Monitoring tells you that something is wrong — error rate is elevated. Observability lets you figure out why — which specific requests are failing, from which users, hitting which service, caused by which code path, correlated with which recent deployment, affecting which percentage of traffic to which model version.

The three foundational principles of observability:

You can't predict every failure mode — complex distributed systems fail in unpredictable ways. The interaction between 15 microservices, under specific load patterns, with specific combinations of input data, produces failure modes that nobody designed for and nobody anticipated. If your monitoring only covers failure modes you thought of, you're blind to everything else. Observability means your system emits rich enough data that even novel failure modes can be investigated.

Instrumentation must be built in, not bolted on — observability is not something you add after a system is built. It's designed into the system from the start. Every service emits structured logs. Every request carries a trace ID. Every key operation is timed and counted. This is part of how the system works, not an afterthought. Retrofitting observability onto a system built without it is painful and incomplete.

High cardinality data is what you actually need — cardinality refers to the number of unique values a field can have. Low cardinality: `status=200` or `status=500` (a handful of values). High cardinality: `user_id=a8f3b2...` or `request_id=9d7c3e...` (millions of possible values). Most traditional monitoring systems struggle with high cardinality because they store aggregated metrics. But high cardinality is exactly what you need during debugging — you don't want to know that 0.1% of requests failed, you want to know which specific requests failed, from which users, and trace them individually. Observability tooling is designed to handle high cardinality data that traditional metrics systems can't.

The three pillars — metrics, logs, and traces are the three primary data types that feed observability. Each answers different questions. Metrics tell you what is happening and when. Logs tell you what happened in detail. Traces tell you how a request flowed through your system. We'll cover each in depth in the next section.

For ML systems, observability has special dimensions beyond standard software systems. Your model's behavior needs to be observable — not just "did the service return 200 OK?" but "what was the model's prediction confidence distribution?", "are input features drifting from training distribution?", "which user segments are getting different prediction quality?". Model observability is a discipline in itself that sits on top of the infrastructure observability layer.

02 Metrics vs Logs vs Traces

These three data types are the pillars of observability. Each has distinct characteristics — how it's generated, how it's stored, how it's queried, what it's good for, and what it can't tell you. Using all three together gives you a complete picture. Over-relying on any one leaves blind spots.

Metrics are numerical measurements collected at regular intervals and stored as time series. A metric has a name, a value, a timestamp, and a set of labels (key-value pairs that provide context). Examples:

```
http_requests_total{method="POST", status="200", endpoint="/predict"} ,
model_inference_latency_seconds{model_version="v2.1", percentile="p99"} ,
kubernetes_pod_cpu_usage{pod="model-server-abc123", namespace="production"} .
```

Metrics are aggregated data — instead of storing one data point per request, you store the count, sum, average, or distribution across many requests. This aggregation makes metrics extremely efficient to store and query. Querying 6 months of request rate data takes milliseconds because it's stored as aggregated time series, not millions of individual request records.

Metrics are excellent for: dashboards showing overall system health, alerting on threshold violations (error rate above 1%), capacity planning, long-term trend analysis, and SLO tracking. They're terrible for: understanding why a specific request failed (you only have aggregates, not individual data points), debugging complex interactions, and any question that requires high cardinality data.

Logs are timestamped records of discrete events that happened in your system. Every time something notable occurs — a request arrives, an exception is thrown, a model is loaded, a database query is executed — your application writes a log entry. Logs can be unstructured (plain text) or structured (JSON with defined fields).

Structured logging is vastly superior:

```
{
  "timestamp": "2024-01-15T14:23:41.123Z",
  "level": "ERROR",
  "service": "model-server",
  "version": "v2.1.0",
  "request_id": "req-9f7c3e2a",
  "user_id": "usr-a8f3b291",
  "model_version": "v2.1",
  "input_tokens": 847,
  "error": "CUDA out of memory",
  "stack_trace": "...",
  "latency_ms": 1203
}
```

Structured logs are machine-parseable. You can filter all errors from model version v2.1, group errors by type, correlate specific requests by request_id, and join log events across services using a shared trace ID. Unstructured text logs require regex parsing and are fragile.

Logs are excellent for: understanding exactly what happened during a specific request, debugging errors by reading stack traces and context, audit trails of user actions and system changes, and detailed diagnosis of specific failures. They're expensive compared to metrics — storing every log event for months requires significant storage — and querying across large log volumes can be slow.

Traces are records of a request's journey through a distributed system. In a microservices architecture, a single user request might touch 8 services before returning a response. A trace captures the entire journey — every service hop, the time spent in each, the operations performed, and any errors that occurred.

A trace consists of spans. A span represents a unit of work — one HTTP call, one database query, one model inference. Each span has a span ID, a parent span ID (except the root span), start time, duration, status, and attributes. All spans sharing a trace ID belong to the same trace.

Traces are excellent for: understanding end-to-end latency and which service is responsible for slowness, identifying cascading failures, understanding complex request flows, and debugging performance problems in distributed systems. They answer questions like "why did this specific request take 4 seconds?" — you can see that it spent 3.2 seconds waiting for the feature store and 0.8 seconds in model inference.

Using all three together — the power comes from correlation. Your metrics alert fires: p99 latency elevated for the predict endpoint. You look at your dashboard, narrow the time window to the spike. You query logs for the error trace IDs during that window. You open a specific trace to see that requests spent 3 seconds in the feature store. You look at feature store metrics and see connection pool exhaustion at that timestamp. Root cause: feature store connection pool too small for current traffic. Each pillar answered a different piece of the puzzle.

The concept of golden signals comes from Google's Site Reliability Engineering (SRE) book. Google observed that across virtually all services — web services, databases, batch jobs, ML serving — there are four metrics that, together, tell you whether your service is healthy and if not, why not. Monitoring these four signals gives you the highest return on observability investment.

Latency is how long it takes to service a request. This must be measured as a distribution, not an average. Averages are deeply misleading for latency because they hide the tail. If 99% of requests complete in 50ms and 1% take 30 seconds, the average might be 350ms — which looks acceptable but represents a terrible user experience for 1% of your users. Always measure and alert on percentiles: p50 (median), p95, p99, p99.9. The right percentile to alert on depends on your user expectations.

Latency must be measured separately for successful requests and failed requests. A request that fails immediately has very low latency — including it in your success latency metrics makes your latency look artificially good during incidents when many requests are failing fast.

For ML systems, latency has special subcomponents worth measuring separately: feature retrieval latency (how long to get features from the feature store), model inference latency (the actual model forward pass), and pre/post-processing latency. When inference is slow, you need to know whether the model itself is slow or the surrounding infrastructure is.

Traffic is a measure of demand on your system. Requests per second is the most common measure, but the right metric depends on your service type. For a streaming service, it might be active connections. For a batch ML training system, it might be jobs queued and running. For a model serving API, it's requests per second and perhaps also requests by model version.

Traffic is the denominator for error rate calculations — you can only interpret absolute error counts in the context of traffic volume. 100 errors per minute during 10,000 requests per minute (1%) is very different from 100 errors per minute during 110 requests per minute (91%). Traffic is also essential for capacity planning and understanding when you're approaching your system's limits.

Errors is the rate of requests that are failing. This includes both explicit failures (HTTP 500 responses, exceptions thrown, predictions returning null) and implicit failures (HTTP 200 responses that return wrong data, predictions that are technically valid but nonsensical). The explicit errors are easier to measure — your instrumentation can count non-200 response codes. Implicit errors require business logic to detect — a prediction confidence below 0.1% might indicate the model is generating garbage even though the request technically succeeded.

Error rate, calculated as errors divided by total requests over a time window, is your primary signal that something is functionally wrong. Most SLOs are expressed in terms of error rate. The distinction between client errors (4xx — the caller is sending bad requests) and server errors (5xx — your service is broken) matters for alerting — you should alert on server errors but not necessarily on client error rates that are beyond your control.

Saturation is how full or overloaded your service is. It's a measure of how much of your capacity you're using. CPU utilization, memory usage, GPU utilization for ML inference, database connection pool utilization, queue depth — these are all saturation metrics. When saturation approaches 100%, latency increases and errors often follow. Saturation is a leading indicator — it warns you that problems are coming before they arrive, giving you time to scale before users are affected.

For ML systems, GPU utilization and GPU memory are critical saturation metrics. A model server at 95% GPU memory is close to OOMKilling under load. A training job at 40% GPU utilization is wasting expensive resources. Saturation metrics for GPU workloads guide both incident response and capacity optimization.

The insight behind golden signals — you don't need 200 different metrics to understand whether your service is healthy. You need these four, measured well, displayed clearly, and alerted on thoughtfully. If all four golden signals are in their normal ranges, your service is almost certainly fine. If any is abnormal, you have a clear starting point for investigation.

04 Prometheus Architecture

Prometheus is the de facto standard monitoring system in the Kubernetes and cloud-native ecosystem. Understanding its architecture deeply is essential for anyone building production ML infrastructure, because you will encounter Prometheus in virtually every Kubernetes environment.

Prometheus is a time series database combined with a metrics collection engine and a query language. Its architecture is deliberately simple, which is part of why it's so widely adopted.

The pull model — Prometheus collects metrics by pulling (scraping) them from targets, rather than having targets push metrics to Prometheus. This is the opposite of many traditional monitoring systems. Prometheus periodically makes HTTP GET requests to `/metrics` endpoints on each configured target. Each target responds with the current values of all its metrics in Prometheus's text exposition format. Prometheus stores these samples in its time series database.

The pull model has several advantages. You can tell from the scraper whether a target is up or down — if the scrape fails, the target is probably unhealthy. You have central control over what's scraped and how often. You can easily test what a target is exposing by hitting its `/metrics` endpoint in a browser or with curl. The disadvantage is that very short-lived targets (batch jobs that complete before the next scrape) might be missed — Prometheus has a separate component called Pushgateway for these.

Scrape targets are discovered statically (you list target addresses in the config) or dynamically through service discovery. Kubernetes service discovery is the most important for our context — Prometheus can automatically discover all pods, services, and nodes in a Kubernetes cluster and scrape the ones that are configured to expose metrics (indicated by Kubernetes annotations). When a new Pod starts, Prometheus discovers it automatically.

The data model — every metric has a name and a set of labels (key-value pairs). Labels are what give Prometheus its power and flexibility. `http_requests_total` is a metric name. `{method="POST", handler="/predict", status="200"}` are labels. The combination of a metric name and a specific set of label values is called a time series. Prometheus stores the value of each time series at each scrape interval.

The four metric types Prometheus supports: **Counter** (a value that only goes up, like total request count), **Gauge** (a value that can go up and down, like current memory usage), **Histogram** (records a distribution of observations into configurable buckets, like request latency), and **Summary** (similar to histogram but calculates percentiles client-side).

Histograms are particularly important for latency monitoring because they let you calculate percentiles server-side using Prometheus's `histogram_quantile` function. Measuring p99 latency over a time window is a Histogram query.

PromQL (Prometheus Query Language) is the query language for analyzing stored metrics. It's powerful and somewhat unusual because it operates on time series natively. Key operations:

promql

15 lines

```
# Rate of HTTP requests per second over 5-minute window
rate(http_requests_total[5m])

# P99 latency from histogram data
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

# Error rate as a ratio
sum(rate(http_requests_total{status=~"5.."}[5m])) /
sum(rate(http_requests_total[5m]))

# ML model inference latency by model version
histogram_quantile(0.95,
  rate(model_inference_duration_seconds_bucket{namespace="production"}[5m])
) by (model_version)
```

Alertmanager is a separate component that receives alerts from Prometheus (when an alerting rule's condition is true), deduplicates them (if the same alert fires from multiple Prometheus instances), groups related alerts together, applies silences, and routes alerts to notification receivers (PagerDuty, Slack, email, OpsGenie). Alertmanager handles the "what to do when an alert fires" problem while Prometheus handles "when should an alert fire."

Prometheus Operator and kube-prometheus-stack — in Kubernetes, the Prometheus Operator is the standard way to deploy and manage Prometheus. It introduces Kubernetes Custom Resources like `ServiceMonitor` and `PodMonitor` that define which services to scrape, making Prometheus configuration itself declarative and Kubernetes-native. The kube-prometheus-stack Helm chart deploys the Prometheus Operator, Prometheus, Alertmanager, Grafana, and a comprehensive set of pre-built dashboards and alerting rules for Kubernetes cluster monitoring in a single command.

05 Metrics Instrumentation

Instrumentation is the practice of adding metrics collection code to your application so that it emits data about its behavior. Good instrumentation is what makes your system observable — without it, Prometheus has nothing to scrape and your dashboards are empty.

The golden rule of instrumentation: measure what matters to users, not what's easy to measure. CPU usage is easy to measure but doesn't directly tell you whether your model server is giving users good predictions. Inference latency and prediction quality directly measure user experience. Start with what matters, add lower-level metrics for diagnosis.

Python instrumentation with prometheus-client:

python

53 lines


```

from prometheus_client import Counter, Histogram, Gauge, start_http_server
import time

# Counter: only goes up, tracks total occurrences
INFERENCE_REQUESTS = Counter(
    'model_inference_requests_total',
    'Total number of inference requests',
    ['model_version', 'status'] # Labels
)

# Histogram: records distribution of values (latency, sizes)
INFERENCE_LATENCY = Histogram(
    'model_inference_duration_seconds',
    'Time spent running model inference',
    ['model_version'],
    buckets=[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0]
)

# Gauge: can go up and down (current value)
ACTIVE_REQUESTS = Gauge(
    'model_server_active_requests',
    'Number of requests currently being processed'
)

MODEL_CONFIDENCE = Histogram(
    'model_prediction_confidence',
    'Distribution of prediction confidence scores',
    ['model_version'],
    buckets=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99, 1.0]
)

def run_inference(request, model_version):
    ACTIVE_REQUESTS.inc()
    start_time = time.time()

    try:
        result = model.predict(request.features)

        duration = time.time() - start_time
        INFERENCE_LATENCY.labels(model_version=model_version).observe(duration)
        INFERENCE_REQUESTS.labels(model_version=model_version, status='success').inc()
        MODEL_CONFIDENCE.labels(model_version=model_version).observe(result.confidence)

        return result
    except Exception as e:
        INFERENCE_REQUESTS.labels(model_version=model_version, status='error').inc()
        raise
    finally:
        ACTIVE_REQUESTS.dec()

```



```
# Expose metrics on /metrics endpoint
start_http_server(8000)
```

Labeling strategy — labels are powerful but come with cost. Each unique combination of label values creates a new time series in Prometheus. If you label by `user_id`, and you have 10 million users, you create 10 million time series — this is called a cardinality explosion and can crash Prometheus. Labels should have bounded cardinality. Good labels: `model_version` (a few values), `status` (success/error/timeout), `endpoint` (/predict, /health). Bad labels: user IDs, request IDs, input text (unbounded cardinality).

USE method for resources — for every resource (CPU, memory, GPU, network, disk), instrument Utilization (how much of the resource is being used), Saturation (how much work is queued/waiting for the resource), and Errors (error events for the resource). This gives you a complete picture of resource health.

RED method for services — for every service, instrument Rate (requests per second), Errors (failed requests per second), and Duration (distribution of request latency). The RED method is the service-level complement to the resource-level USE method, and together they give a complete picture of system health. Rate, Errors, and Duration map directly to three of the four golden signals.

ML-specific metrics — beyond standard request metrics, ML systems should instrument: feature retrieval latency and errors, model load time (how long to load weights into memory), input validation failures (malformed inputs reaching the model), prediction distribution statistics (mean confidence, distribution of output values), data drift metrics (statistical distance between current input distribution and training distribution), and model-specific business metrics (click-through rate for recommendation models, conversion rate for propensity models).

06 Grafana Dashboards

Grafana is the standard visualization layer for Prometheus (and many other data sources). It turns raw time series data into dashboards — collections of visualizations that give operational teams a real-time view of system health. A well-designed dashboard can reduce mean time to detection from minutes to seconds. A poorly designed dashboard is ignored.

Understanding Grafana starts with understanding its components.

Data sources — Grafana connects to external data stores (Prometheus, Loki, Elasticsearch, CloudWatch, InfluxDB, and many more) and queries them on demand. You configure data sources with connection details, and then every panel in every dashboard queries one of your configured data sources. Prometheus is the most common data source, but a full observability stack might also have Loki for logs and Tempo for traces — all in the same Grafana instance, allowing you to navigate between metrics, logs, and traces in a unified interface.

Panels are individual visualizations within a dashboard. Each panel has a query (PromQL for Prometheus data sources), a visualization type, and display options. Common panel types:

Time series panels show how a metric changes over time — the classic line graph. Use for: request rate over time, latency trends, error rate evolution. Every time series panel on your dashboard should have meaningful Y-axis units (seconds not just numbers, requests/second not just rate), a legend that explains what each line is, and appropriate thresholds (red line at your SLO violation level).

Stat panels show a single current value prominently. Use for: current error rate, current p99 latency, current active connections. Stat panels are great for the "at a glance" overview section at the top of a dashboard.

Gauge panels show a current value relative to a defined range, like a speedometer. Use for: CPU utilization percentage, disk usage percentage, GPU memory utilization.

Table panels show structured data in rows and columns. Use for: top 10 slowest endpoints, list of current incidents, model version deployment status.

Heatmap panels show distribution data over time — how latency distribution changes throughout the day. Use for: latency histograms, prediction confidence distributions.

Dashboard design principles:

Design for the 3am scenario. When someone is paged at 3am and opens your dashboard, they should be able to answer "is there a problem and how bad is it?" within 10 seconds, and "where is the problem?" within 60 seconds. Put the most important signals at the top. Use color consistently — red is bad, green is good, yellow is warning. Avoid dashboard sprawl — one well-designed dashboard beats ten dashboards of individual metrics.

Use template variables for filtering. A model server dashboard should have a variable for `model_version` that filters all panels. A variable for `namespace` that lets you switch between staging and production. Variables are configured in dashboard settings and propagate to all panel queries automatically.

promql

8 lines

```
# Panel query using template variables
histogram_quantile(0.99,
  rate(model_inference_duration_seconds_bucket{
    namespace="$namespace",
    model_version="$model_version"
  }[5m])
)
```

Annotate your dashboards with deployment events. Grafana supports annotations — vertical lines marking specific events in time. Whenever a deployment happens, you add a Grafana annotation. Operators can immediately see correlations between "we deployed at 2pm" and "latency spiked at 2:03pm" without having to check multiple systems.

Grafana alerting — Grafana can directly create alerts from panel queries, routing alerts through Alertmanager or its own notification system. This lets you define alerts visually in the same place you define dashboards, and preview what they would have triggered historically on your time series graph.

07 Centralized Logging

In a modern microservices or Kubernetes environment, logs are generated by hundreds of containers running across dozens of nodes. Finding the logs relevant to a specific request or incident means looking in the right place at the right time. Without centralized logging, this means SSH-ing into individual nodes, running `kubectl logs` for specific pods, manually correlating output from multiple sources. This is tedious at best and impossible during fast-moving incidents.

Centralized logging means all logs from all services and infrastructure are shipped to a single, searchable, queryable system. Regardless of which node a Pod is running on, you can search across all logs from all services for the time window of interest, filter by service, by log level, by error type, by user ID, by request ID — anything in the log's structured fields.

The logging pipeline has three stages: collection, shipping, and storage/querying.

Collection happens at the source. Applications write logs to stdout (the Kubernetes-recommended approach — each container writes its logs to stdout, and the container runtime captures them). The container runtime writes these logs to files on the node. Collection agents running on each node (as DaemonSets in Kubernetes) pick up these files and prepare them for shipping. Alternatively, applications can directly write to a remote log store using a logging library configured with the appropriate backend.

Shipping moves logs from the collection points to centralized storage. Fluentd and Fluent Bit are the most widely used log shippers in Kubernetes environments. Fluent Bit is lighter and recommended for the per-node collection agent (it runs on every node, so resource efficiency matters). Fluentd is more feature-rich and used for aggregation and transformation. The Vector log pipeline is an increasingly popular modern alternative.

Shippers handle buffering (to handle temporary network outages without losing logs), parsing (extracting structured fields from log text, transforming formats), enrichment (adding metadata like Kubernetes pod name, namespace, node, cluster name from the container runtime's metadata), and routing (sending different log streams to different destinations — audit logs to one place, application logs to another).

Storage and querying — the central log store must support full-text search across enormous volumes of data. Common options:

Elasticsearch is the traditional choice — a distributed search and analytics engine that indexes logs for fast full-text and structured queries. Kibana is Elasticsearch's visualization companion. The ELK stack (Elasticsearch, Logstash, Kibana) or EFK (Elasticsearch, Fluent Bit, Kibana) are classic centralized logging architectures.

Loki is Grafana Labs' log aggregation system designed specifically for Kubernetes environments. Unlike Elasticsearch, Loki doesn't index log content — it indexes only the metadata labels (pod name, namespace, service). Logs are stored compressed in object storage (S3, GCS). Queries filter by label first, then search within the matching log streams. This makes Loki dramatically cheaper to operate than Elasticsearch (no expensive indexing of every word), at the cost of slower full-text search. For most use cases, filtering by service and time window and then searching within that filtered set is fast enough.

Loki integrates natively with Grafana, letting you navigate from a metrics alert in Grafana directly to the relevant Loki logs in the same interface, connected by the same time window.

Structured logging is essential — centralized logging's value multiplies with structured logs. If all your logs are unstructured text, searching for all errors from the model server in the last hour requires text pattern matching. If your logs are structured JSON with a `level` field and a `service` field, it's a precise filter: `{service="model-server"} |> "level=error"`. The difference in reliability and speed is enormous.

08 Log Aggregation Concepts

Log aggregation goes beyond just centralizing logs — it's about transforming, enriching, correlating, and routing logs to extract maximum value from the data being emitted.

Parsing and structured extraction — many application logs are text with embedded structured data. A log line like `2024-01-15 14:23:41 ERROR model_server request_id=abc123 user_id=usr-456 latency_ms=1203 error="CUDA OOM"` contains valuable fields but they're embedded in text. Log shippers parse this into a structured record with actual fields: `request_id: "abc123"`, `user_id: "usr-456"`, `latency_ms: 1203`, `error: "CUDA OOM"`. Then you can filter, aggregate, and visualize by any of these fields.

Grok patterns (for Fluentd), VRL (for Vector), and Logstash filter plugins are all mechanisms for parsing unstructured log text into structured records. But the better solution is for applications to emit structured JSON logs in the first place, eliminating the need for complex parsing.

Metadata enrichment — when Fluent Bit collects a log from a container, it knows the Pod name, namespace, container name, and node name from the Kubernetes API. Enriching every log record with this metadata means you can filter logs by Kubernetes context without the application needing to know its own Kubernetes identity.

Log correlation — the most powerful log analysis links log records from different services that handled the same request. This is done through a shared correlation ID (trace ID, request ID) that travels with the request through every service. When the API gateway receives a request, it generates a request ID and adds it to the request headers. Every service that handles the request includes this ID in its log records. In the centralized log system, searching for a specific request ID returns all log records from all services that touched that request, in chronological order, giving you the complete story of what happened.

This only works if every service actually propagates and logs the correlation ID — it requires consistent implementation across all services. OpenTelemetry (covered later) provides standardized propagation so the trace ID automatically flows through HTTP and gRPC calls.

Log sampling — at high traffic volumes, logging every request at DEBUG level produces an overwhelming volume of data that's expensive to store and slow to query. Log sampling means you log a representative subset of requests in full detail rather than all of them. Different strategies: random sampling (log 1% of all requests), head-based sampling (decide whether to log when the request arrives and log all records for sampled requests), tail-based sampling (buffer logs and decide whether to log at the end of the request based on whether it was interesting — slow, errored, or high-value).

For ML systems, log all errors and all slow requests (latency above p99 threshold), and sample normal requests. Log all requests from specific high-value user segments or requests to specific model versions under test. This ensures you have complete data for debugging failures and good statistical data for understanding normal behavior, without storing logs for every routine successful prediction.

09 Distributed Tracing Basics

When a user request to your ML platform touches the API gateway, the model router, the feature store, the model server, and the monitoring system before returning a response, and that response is slow, a stack trace doesn't help you. Stack traces are for single-process debugging. Distributed tracing is for multi-service debugging.

A trace represents the entire lifecycle of a request as it propagates through multiple services. It's structured as a tree of spans, where each span represents a unit of work — processing in one service, a database call, an HTTP request to another service, a model inference call.

Every span has: - A trace ID (same across all spans in the trace) - A span ID (unique to this span) - A parent span ID (the span that caused this span, except for the root span) - An operation name (what work this span represents) - A start timestamp and duration - Status (success or error) - Attributes (key-value pairs with additional context) - Events (timestamped annotations within the span's duration)

diagram

14 lines

```
Trace: req-9f7c3e2a (total: 847ms)
├── API Gateway handler (12ms)
│   ├── Authentication check (3ms)
│   └── Route to model router (2ms)
├── Model Router (823ms)
│   ├── Feature retrieval (621ms) ← This is why it's slow
│   │   ├── Cache lookup (3ms) [MISS]
│   │   └── Feature store query (617ms)
│   ├── Model inference (198ms)
│   │   ├── Input preprocessing (12ms)
│   │   ├── Forward pass (178ms)
│   │   └── Output postprocessing (8ms)
└── Response serialization (12ms)
```

Reading this trace, you immediately see that feature retrieval took 621ms of the 847ms total — specifically a cache miss followed by a slow feature store query. Without this trace, you'd know the request was slow but have no idea where to look.

Context propagation — for spans to form a coherent trace across service boundaries, trace context (trace ID and parent span ID) must travel with the request. When Service A makes an HTTP call to Service B, it adds the trace context to the request headers. Service B extracts the context, creates a child span, and propagates the context when calling Service C. This chain of propagation is what links spans from different services into a single trace.

The W3C Trace Context standard (`traceparent` and `tracestate` headers) is the modern, interoperable standard for HTTP trace context propagation. B3 propagation (from Zipkin) is older and still widely encountered. OpenTelemetry supports both.

Trace storage and querying — traces are stored in dedicated trace backends. Jaeger is the most widely used open-source option. Zipkin is older and simpler. Tempo (from Grafana Labs) stores traces in object storage and integrates natively with Grafana, allowing navigation from metrics or logs directly to relevant traces. Cloud options include AWS X-Ray, Google Cloud Trace, and Datadog APM.

Trace backends let you search for traces by: service name, operation name, duration (find all traces slower than 2 seconds), status (find all error traces), and attribute values (find all traces with `model_version=v2.1`). This last capability — searching by arbitrary attribute — is high-cardinality querying that metrics can't support.

Sampling — storing every trace for high-traffic systems would be prohibitively expensive. Most production tracing systems sample — only a fraction of requests generate full traces. Head-based sampling decides at the start of a request whether to trace it (simple but may miss slow or failing requests). Tail-based sampling

buffers trace data and decides after the request completes, ensuring you always trace slow and failed requests regardless of sampling rate. Tail-based sampling is more complex but produces much more useful trace data.

10 OpenTelemetry Fundamentals

Before OpenTelemetry, the observability tool landscape was fragmented. Different APM vendors had different instrumentation libraries, different data formats, different propagation protocols. If you instrumented your application with Datadog's tracing library and then wanted to switch to Jaeger, you had to re-instrument your entire application. Vendor lock-in was a significant problem.

OpenTelemetry (OTel) is a CNCF project that provides a standardized, vendor-neutral specification and implementation for generating, collecting, and exporting observability data — metrics, logs, and traces. It's now the industry standard for instrumentation and is supported by every major observability vendor.

The key components of OpenTelemetry:

The specification defines the data models (what a span looks like, what a metric looks like), the APIs (how application code generates telemetry), and the semantic conventions (what attributes to use for common concepts — HTTP requests should use `http.method`, `http.url`, `http.status_code`; database calls should use `db.system`, `db.name`). Semantic conventions ensure that data from different services and different languages uses consistent attribute names, enabling cross-service analysis.

SDKs implement the specification in specific languages — Python, Java, Go, JavaScript, .NET, and more. The SDK handles span creation, metric recording, context propagation, sampling decisions, and exporting. Once you initialize the OTel SDK in your application, it provides the APIs your code uses to create spans and record metrics.

Auto-instrumentation is one of OpenTelemetry's most powerful features. For common frameworks and libraries, OTel provides automatic instrumentation — you configure it once and it automatically generates spans for every incoming HTTP request, every outgoing HTTP call, every database query, every message queue operation. For Python, the `opentelemetry-auto-instrumentation` package instruments Flask, FastAPI, requests, SQLAlchemy, Redis, and many other popular libraries without code changes.

python

29 lines

```
# Manual instrumentation example
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter

# Initialize the tracer
provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(OTLPSpanExporter(endpoint="http://otel-collector:4317"))
)
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("model-server")
```

```
def run_inference(features, model_version):
    with tracer.start_as_current_span("model_inference") as span:
        span.set_attribute("model.version", model_version)
        span.set_attribute("input.feature_count", len(features))

        with tracer.start_as_current_span("feature_validation"):
            validated = validate_features(features)

            with tracer.start_as_current_span("forward_pass") as fp_span:
                result = model.predict(validated)
                fp_span.set_attribute("prediction.confidence", result.confidence)

            span.set_attribute("prediction.label", result.label)
    return result
```

The OpenTelemetry Collector is a standalone service that receives telemetry data from applications, processes it (filtering, transforming, batching), and exports it to multiple backends. Rather than configuring each service to export directly to Jaeger, Prometheus, and Loki separately, services export to the Collector, and the Collector routes to all backends. This decouples instrumentation from backend choice — you can add or change backends by reconfiguring the Collector without touching application code.

The Collector pipeline: receivers (accept incoming data in various formats — OTLP, Prometheus, Jaeger, Zipkin), processors (transform data — add attributes, sample, batch), exporters (send to backends — Jaeger, Prometheus, Tempo, Loki, Datadog, Grafana Cloud).

For ML systems, OpenTelemetry enables end-to-end tracing from the user API request through model routing, feature retrieval, model inference, and back. By using OTel from the beginning, you avoid vendor lock-in — you can switch from Jaeger to Tempo to Datadog APM by changing Collector configuration, not application code. The semantic conventions are evolving to cover ML-specific operations, making ML telemetry more consistent and queryable across tools.

11 Alerting & Incident Response

Alerting is the mechanism that tells you something is wrong in your system. Done well, alerts wake you up only when something genuinely requires attention. Done poorly, alerts wake you up constantly for things that don't need human intervention — leading to alert fatigue, where engineers start ignoring alerts because so many are false positives, and then miss the ones that matter.

Alert design principles:

Every alert should be actionable. If there's nothing a human can do when an alert fires, it shouldn't be an alert — it should be a dashboard metric to observe or a recording rule to track. An alert that fires for "CPU above 70%" on a node that auto-scales is not actionable — the system handles it automatically. An alert that fires for "all pods in the model server deployment are crash-looping" is actionable — a human needs to investigate immediately.

Alert on symptoms, not causes. Alert on elevated error rates (the symptom users experience), not on the specific CPU spike or memory pressure that might be causing it (a cause that might not actually affect users). Cause-based alerting produces many alerts for the same user-visible problem, creating noise.

Symptom-based alerting produces one alert per user-visible problem, creating clarity.

Tune alert thresholds based on data, not intuition. "Alert when error rate exceeds 1%" sounds reasonable but might fire constantly in a system that normally has 0.8% errors. Look at your historical data, understand normal operating ranges, and set thresholds outside the normal range.

Alert severity levels give responders immediate context for urgency:

Critical/P1 — user impact, needs immediate response 24/7. Model serving is completely down. Error rate above SLO. These should be rare. Paged immediately regardless of time.

High/P2 — significant impact, needs response within hours. Elevated error rate below SLO threshold but trending up. One of N replicas is failing. Business hours response acceptable.

Warning/P3 — potential future problem, needs response within a day or two. Disk space filling up, memory usage growing over days, queue depth slowly increasing. Proactive attention before it becomes critical.

Prometheus alerting rules define conditions that trigger alerts:

yaml

30 lines

```
groups:
- name: model-server-alerts
  rules:
  - alert: ModelServerHighErrorRate
    expr: |
      (
        sum(rate(model_inference_requests_total{status="error"}[5m]))
        /
        sum(rate(model_inference_requests_total[5m]))
      ) > 0.01
    for: 5m # Must be true for 5 minutes before firing (prevents flapping)
    labels:
      severity: critical
      team: ml-platform
    annotations:
      summary: "Model server error rate above 1%"
      description: "Error rate is {{ $value | humanizePercentage }} over the last 5 minutes"
      runbook_url: "https://wiki.company.com/runbooks/model-server-high-errors"

  - alert: ModelInferenceP99LatencyHigh
    expr: |
      histogram_quantile(0.99,
        rate(model_inference_duration_seconds_bucket[5m]))
      ) > 2.0
    for: 10m
    labels:
      severity: high
    annotations:
      summary: "Model inference P99 latency above 2 seconds"
```


The `for` clause is critical — it prevents alerts from firing on momentary spikes. The condition must be continuously true for the specified duration before the alert fires. This dramatically reduces false positives from brief anomalies.

Runbooks are the documentation that tells an on-call engineer what to do when a specific alert fires. A good runbook contains: what the alert means, what the impact is, immediate diagnostic steps (what to look at first), likely causes and their solutions, escalation path if the on-call engineer can't resolve it, and links to relevant dashboards and documentation. Runbooks should be linked directly from alert definitions so the on-call engineer can navigate there immediately from the alert notification.

Incident response process — when a critical alert fires:

Detection: alert fires, on-call engineer is paged.

Triage: what is the user impact? How many users affected? How severe? This determines urgency.

Communication: for significant incidents, open an incident channel and start a status page update. Internal stakeholders need to know even before resolution. "We're investigating elevated error rates on the model serving API" is better than silence.

Investigation: use your observability stack. Dashboard → metrics → logs → traces. Follow the data. The golden signals point to what's wrong. Logs and traces explain why.

Mitigation: restore service as quickly as possible, even if imperfectly. Roll back the last deployment if it's deployment-related. Scale up if it's capacity-related. Disable a broken feature flag. Perfection can wait for post-incident.

Resolution: confirm service is restored and stable. Update status page.

Post-incident review (blameless post-mortem): what happened? What was the impact? What worked well in the response? What didn't? What would prevent this? Write it up within 48 hours while memory is fresh. The output is action items that reduce future incidents.

12 Monitoring Kubernetes Workloads

Kubernetes adds a layer of complexity to monitoring because you have two distinct layers to observe: the Kubernetes infrastructure layer (cluster components, nodes, pods, deployments) and the application layer (your actual services running on Kubernetes). Both need monitoring and they interact — an unhealthy node affects the applications on it, an OOMKilled pod is both a Kubernetes event and an application failure.

Kubernetes infrastructure monitoring covers the components that make up the cluster itself.

Node metrics are fundamental. Each node's CPU, memory, disk, and network utilization determine whether there's capacity for workloads. `node_cpu_seconds_total`, `node_memory_MemAvailable_bytes`, `node_disk_io_time_seconds_total` from the Node Exporter (a Prometheus exporter that runs on each node and exposes hardware and OS-level metrics) provide this data. Alerts on high node memory utilization prevent OOMKills before they happen.

Control plane metrics show the health of Kubernetes itself. API server request latency and error rates indicate whether kubectl and controllers can communicate reliably. etcd latency and size indicate database health. Scheduler throughput shows whether Pods are being scheduled promptly.

Pod and workload metrics from the Kubernetes API itself: pod restarts (high restart counts indicate crash loops), pending pod count (pods stuck pending indicate scheduling problems — insufficient resources, affinity conflicts), deployment rollout status, node readiness.

kube-state-metrics is a critical component that exposes Kubernetes object state as Prometheus metrics. It queries the Kubernetes API and exposes metrics like `kube_deployment_status_replicas_available`, `kube_pod_container_status_restarts_total`, `kube_pod_status_phase`, `kube_node_status_condition`. These are metadata about Kubernetes resources — not resource utilization (that's Node Exporter and cAdvisor) but the logical state of objects.

promql

11 lines

```
# Alert when a deployment has fewer ready replicas than desired
kube_deployment_status_replicas_ready

kube_deployment_spec_replicas

# Alert on excessive pod restarts
increase(kube_pod_container_status_restarts_total[1h]) > 5

# Pending pods - might indicate resource pressure
kube_pod_status_phase{phase="Pending"} > 0
```

Application monitoring on Kubernetes — your application containers should expose Prometheus metrics on a `/metrics` endpoint, and your Prometheus instance should scrape them. Using Prometheus Operator's `ServiceMonitor` resources makes this declarative:

yaml

17 lines

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: model-server-metrics
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: model-server
  namespaceSelector:
    matchNames:
      - ml-production
  endpoints:
    - port: metrics
      interval: 15s
      path: /metrics
```

This tells Prometheus Operator to scrape any service with label `app: model-server` in the `ml-production` namespace every 15 seconds. When you deploy a new model server service with that label, it's automatically discovered and scraped.

GPU monitoring for ML workloads — NVIDIA provides a Prometheus exporter called `dcgm-exporter` (NVIDIA Data Center GPU Manager Exporter) that exposes detailed GPU metrics. Running this as a DaemonSet on GPU nodes exposes: GPU utilization percentage, GPU memory used and free, GPU temperature, GPU power consumption, and GPU error counts. These are essential for understanding whether your model serving and training workloads are using GPU resources efficiently and for detecting hardware issues.

promql

9 lines

```
# GPU memory utilization across all GPU nodes
DCGM_FI_DEV_MEM_COPY_UTIL

# Alert on GPU memory nearly full
DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL > 0.9

# GPU temperature alert
DCGM_FI_DEV_GPU_TEMP > 85
```

Namespace-level resource usage — for multi-tenant clusters where different teams share resources, monitoring resource usage per namespace enables fair accounting and prevents surprise resource exhaustion:

promql

10 lines

```
# CPU usage by namespace
sum by (namespace) (
  rate(container_cpu_usage_seconds_total[5m])
)

# Memory usage by namespace
sum by (namespace) (
  container_memory_working_set_bytes
)
```

13 SLOs, SLAs & SLIs

SLOs, SLAs, and SLIs form a framework for defining, measuring, and communicating the reliability of your services in precise, quantitative terms. This framework comes from Google's SRE practice and is now widely adopted across the industry. Understanding this framework is important for production engineers because it changes how you think about reliability — from "is the service up?" to "is the service meeting user expectations?"

SLI (Service Level Indicator) is the actual measurement — a quantitative metric that indicates the level of service being provided. An SLI is a carefully chosen metric that correlates with user experience. Good SLIs measure things users care about, not internal technical metrics that users never see.

Examples of SLIs: - Request success rate: the proportion of valid requests that returned a successful response - Latency: the proportion of requests that completed within a specified threshold - Availability: the proportion of time the system was able to serve requests

For ML systems, SLIs might include: the proportion of inference requests returning a prediction within 500ms, the proportion of training jobs that complete without error, the proportion of model predictions with confidence above a minimum threshold.

SLO (Service Level Objective) is a target value or range for an SLI. It's an internal commitment about the reliability level you aim to provide. An SLO is not a promise to users (that's the SLA) — it's a goal that guides engineering decisions.

Examples of SLOs: - 99.9% of API requests will succeed (success rate SLI $\geq$ 99.9%) - 95% of predictions will complete within 200ms (latency SLI $\geq$ 95% under 200ms) - The model serving API will be available 99.95% of the time

SLOs are not 100% because 100% reliability is impossible and pursuing it is prohibitively expensive. Choosing the right SLO is a product and business decision — what reliability level do users actually need? What can you afford to provide? More nines (99.9% $\rightarrow$ 99.99%) require exponentially more engineering effort.

Error budget is the mathematical consequence of an SLO. If your SLO is 99.9% success rate, your error budget is 0.1% of requests — that's how many errors you're "allowed" to have while still meeting your SLO. In a month with 10 million requests, your error budget is 10,000 failed requests. If you've already burned 8,000 of those errors, you have only 2,000 remaining for the rest of the month.

Error budgets change the conversation about reliability from emotional ("we should have zero errors") to rational ("we have X% of our error budget remaining, should we take the risk of deploying this change?"). A risky deployment that might cause 5,000 errors is fine if you have 50,000 errors of budget remaining. The same deployment is inadvisable if you have only 3,000 remaining. Error budgets make reliability tradeoffs explicit and data-driven.

SLA (Service Level Agreement) is a formal agreement between a service provider and a customer that specifies the consequences if SLOs aren't met — typically financial penalties, service credits, or termination rights. SLAs are external and contractual. SLOs are internal targets. The SLO should be stricter than the SLA — you target 99.95% availability internally so that you have buffer before violating the 99.9% SLA. If your internal target is your SLA target, any failure violates customer agreements.

Implementing SLO monitoring with Prometheus:

```
# Recording rule for success rate SLI over 1-hour windows
- record: job:request_success_rate:ratio_rate1h
  expr: |
    sum(rate(http_requests_total{status!~"5.."}[1h]))
    /
    sum(rate(http_requests_total[1h]))

# Alert when error budget is burning too fast
- alert: ErrorBudgetBurnRateTooHigh
  expr: |
    (
      1 - job:request_success_rate:ratio_rate1h
    ) > (1 - 0.999) * 14.4 # 14.4x burn rate = depletes monthly budget in 2 hours
  for: 5m
  annotations:
    summary: "Error budget burning at 14.4x rate"
```

The burn rate concept is important for SLO alerting. Instead of alerting when error rate exceeds the SLO threshold (which would only fire when you're already in violation), you alert when the error budget is being consumed at a rate that will exhaust the budget before the end of the measurement window. A 14.4x burn rate means you'll exhaust your monthly error budget in 50 hours — alert now so you can fix the problem before the budget is gone.

Balancing reliability and velocity — the most profound insight from the error budget model is that reliability and development velocity are in tension, and error budgets make that tension explicit and manageable. If your error budget is healthy (you've used only 20% of the month's budget with half the month to go), you have room to take risks — deploy that risky migration, try that new feature flag, experiment with a new model architecture. If your error budget is nearly exhausted (you've used 95% of the month's budget with two weeks to go), you should freeze risky changes until the budget recovers. The error budget is the objective arbiter of when to prioritize reliability vs velocity.

The connecting thread through all of them is this: you cannot improve what you cannot measure, and you cannot debug what you cannot observe. Building observable systems from the start — with structured logs, meaningful metrics, distributed traces, clear SLOs, and thoughtful alerting — transforms operations from reactive firefighting into proactive engineering. Every concept here, from the golden signals to error budgets, is a tool for understanding your system deeply enough to make it reliably better over time.